

REGRESIÓN DEL REDSHIFT COSMOLÓGICO MEDIANTE REDES NEURONALES ARTIFICIALES



**Trabajo de Fin de Grado
Física**

Alberto Nieto Cardoso

Dirigido por Pietro Vischia

Codirigido por Georgios Alestas



Universidad de Oviedo



Índice

1	Introducción	1
2	Marco Teórico	3
2.1	Conceptos básicos de machine learning	3
2.2	El redshift en cosmología	18
3	El Conjunto de Datos	23
3.1	Recolección y preparación de datos	23
3.2	Preprocesamiento de los datos	28
4	Modelización del Problema	35
4.1	Estructura global	35
4.2	Explicación de los modelos	38
5	Resultados	43
5.1	Análisis comparativo	43
5.2	Problemáticas	51
5.3	Discusión	55
6	Conclusiones	59
7	Bibliografía	61
8	Glosario	63

Capítulo 1

Introducción

Los modelos de inteligencia artificial han dejado de ser una novedad desde hace algunos años. Tanto es así que, el mundo del machine learning ha estado avanzado a pasos agigantados desde que se descubrió el inmenso potencial que poseía este campo, habiéndose explorado ya una inmensa cantidad de enfoques para entrenar estos modelos para tareas en las que nadie imaginaba que fuesen a tener cabida.

Este trabajo trata de aportar a esta tendencia, partiendo de un problema de gran importancia dentro del mundo de la física teórica y en el cual este enfoque podría tener sumo interés. Este es el de estimar con precisión el redshift cosmológico o z , el cual no deja de ser un escalar que toma valores en un espacio continuo, el cual intentará inferirse a partir del espectro del objeto astronómico a partir de varios modelos.

Estos espectros no son imágenes ni características definidas que toman un valor, como el peso o la altura de una persona, sino que para cada muestra se dispone de dos canales enormes de datos que contienen el flujo o cantidad de luz medido para cada longitud de onda de un cierto objeto.

Nuestra tarea será, pues, construir una red neuronal capaz de hallar los patrones físicos inherentes al redshift dentro de los espectros, ignorando las partes de este que no son aptas para la regresión del redshift y que pueden ser interpretadas como ruido para nuestro modelo. Este ruido conforma la mayoría de los espectros astronómicos, ya que solo ciertas longitudes de onda son útiles para extraer información fiable del parámetro.

A lo largo de este informe se presentará la teoría necesaria y los procedimientos seguidos para llegar al modelo final. Partiremos de la nada, comenzando con la exploración de los modelos más simples y desarrollando a partir de ellos otros más refinados y enfocados a nuestra tarea particular, usando como criterio principal la capacidad de generalización del

modelo a grandes conjuntos de datos.

Finalmente comentaremos los problemas particulares que nos hemos encontrado a lo largo del desarrollo del proyecto y las ventajas que tienen estos métodos a pesar de ellos, desvelando si suponen una alternativa válida y útil frente a los enfoques tradicionales que se utilizan hoy en día.

Se aprovecha este pequeño espacio para comentar que este trabajo está fuertemente respaldado por código (**Jupyter Notebook**, del cual es necesario tener disponibilidad para un correcto seguimiento del proyecto. Este puede ser encontrado en [GitHub](#) y para visualizarlo solo se necesita disponer de un visor de **Python**, como **Visual Studio Code**.

Capítulo 2

Marco Teórico

2.1 Conceptos básicos de machine learning

[El Machine Learning es el] campo de estudio que otorga a las computadoras la capacidad de aprender sin ser programadas explícitamente.

—**Arthur Samuel**, 1959

Se dice que un programa informático aprende de la experiencia E con respecto a una tarea T y una medida de desempeño P , si su rendimiento en T , tal como se mide por P , mejora con la experiencia E .

—**Tom Mitchell**, 1997

La gran mayoría de definiciones, tipologías y mecanismos de implementación en **Python** que presentaremos a continuación han sido extraídos del libro [1, *Deep Learning*], el cual sintetiza de manera muy accesible toda la teoría básica e intermedia de este campo.

Dicho esto, el **machine learning** o aprendizaje automático es una disciplina que se remonta a la década de 1950, cuando se exploraron por primera vez métodos computacionales capaces de emular procesos de aprendizaje humano. Fue durante esa época en la que se definió el concepto básico de perceptrón y se sentaron las bases teóricas de las redes neuronales, aprovechando el auge de los ordenadores digitales para implementar modelos lineales de clasificación.

En la última década, el aprendizaje automático ha vivido una auténtica revolución gracias al desarrollo de arquitecturas especializadas, como los famosos *LLM* o modelos de lenguaje grandes, entrenados con enormes volúmenes de texto para comprender, predecir y generar lenguaje de manera coherente (ChatGPT), y las *redes convolucionales*, que son cruciales para el procesamiento y generación de imágenes (Midjourney). Ambas implementaciones sirven como inspiración para el desarrollo de este trabajo.

2.1.1 Fundamentos

Lo primero que necesitamos definir es qué es un **modelo** y en qué consiste modelizar un problema. Se define un modelo computacional como una representación matemática que establece relaciones entre variables de entrada y salida, permitiendo predecir comportamientos a partir de datos. En el contexto del machine learning, se concibe que un modelo aprende patrones estadísticos de un conjunto de entrenamiento para generalizar sobre ejemplos no vistos.

Este trabajo se centra particularmente en el aprendizaje automático **supervisado**. Esto es, sea el conjunto de **datos de entrenamiento** la colección $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, en la que $\mathbf{x}^{(i)} \in \mathbb{R}^m$ representa las m características de la **muestra** i e $y^{(i)}$ el valor objetivo, se define la **función de pérdida** $\mathcal{L}(\theta) = \ell(f_{\theta}(\mathbf{x}), y)$ como la distancia en cierta *métrica* ℓ entre el valor predicho por el modelo f (con parámetros θ) y el valor objetivo.

El proceso de aprendizaje consiste, pues, en ajustar los parámetros internos de la red minimizando esta función de pérdida, que cuantifica la discrepancia entre las predicciones y las etiquetas verdaderas. A cada iteración de este proceso, equivalente a que la red recorra el conjunto de entrenamiento completo una vez, se le llamará **época**.

Además del conjunto de entrenamiento, durante cada época se puede hacer uso de otro conjunto de datos independientes de **validación**, el cual como su propio nombre indica sirve para validar el rendimiento de la red antes muestras nunca vistas. La evaluación de la red en este conjunto es lo que nos permite poder hacer un seguimiento de la capacidad de regresión del modelo.

Bajo este paradigma, una **red neuronal artificial** (RNA) se concibe como un ensamblaje de unidades de procesamiento, llamadas neuronas artificiales, organizadas en **capas** y conectadas mediante **pesos**. Cada neurona recibe un *vector de entradas*, realiza una combinación lineal de las mismas y aplica una función de activación no lineal para producir una *señal de salida*. Dicho ensamblaje se dispone en una arquitectura en la que las capas se suceden sin realimentación, constituyendo lo que se conoce como *red feedforward* o que solo propaga la información en un sentido.

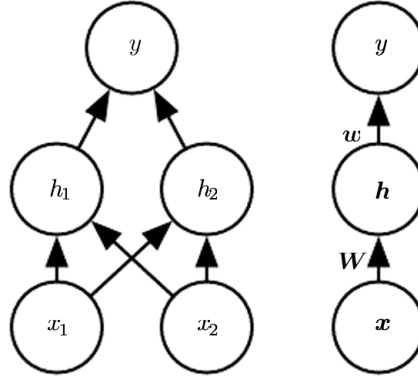


Figura 1: Esquema de la red feedforward en el que aparecen los conceptos de input o entrada $\mathbf{x}$, capa oculta $\mathbf{h}$ y output o salida $\mathbf{y}$, todas unidas mediante los pesos.

Para hablar con propiedad, se deberá distinguir entre **parámetros** del modelo, que corresponden a los pesos y sesgos que se ajustan automáticamente durante el proceso de optimización, e **hiperparámetros**, que deben fijarse antes del entrenamiento y controlan la capacidad de representación y la dinámica de convergencia del algoritmo, como lo pueden ser el número de capas y de neuronas de la red.

Pasamos a definir finalmente la unidad básica de este tipo de modelos. El **perceptrón** constituye el modelo más elemental de una neurona artificial. Se define como una función lineal seguida de una función de activación escalón, de modo que recibe un vector de entrada $\mathbf{x} = [x_1, x_2, \dots, x_n]$ y produce una salida binaria $y \in \{-1, 1\}$.

Matemáticamente, se introduce un vector de pesos $\mathbf{w} = [w_1, w_2, \dots, w_n]$ y un sesgo b , formándose la combinación lineal

$$z = \mathbf{w}^\top \mathbf{x} + b,$$

y aplicando a continuación la función escalón

$$y = \begin{cases} 1, & \text{si } z \geq 0, \\ -1, & \text{si } z < 0. \end{cases}$$

Con esta definición, el perceptrón único separa el espacio de entrada mediante un hiperplano $\mathbf{w}^\top \mathbf{x} + b = 0$, clasificando los puntos de un lado como positivos y del otro como negativos.

El proceso de entrenamiento de este modelo sencillo se basa en la regla de aprendizaje del perceptrón, que ajusta iterativamente los pesos y el sesgo para corregir las clasificaciones erróneas en un conjunto de datos linealmente separable. En cada iteración y para cada muestra $(\mathbf{x}^{(i)}, y^{(i)})$, se actualiza

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y^{(i)} - \hat{y}^{(i)}) \mathbf{x}^{(i)}, \quad b \leftarrow b + \eta (y^{(i)} - \hat{y}^{(i)}),$$

donde $\hat{y}^{(i)}$ es la predicción actual, $y^{(i)}$ la etiqueta verdadera y η la **tasa de aprendizaje**.

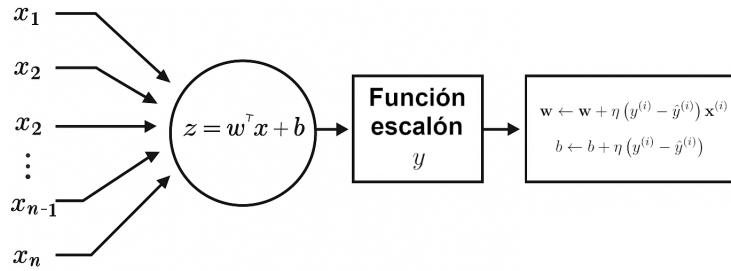


Figura 2: Esquema visual del concepto de perceptrón y su actualización de parámetros.

2.1.2 El perceptrón multicapa

El **perceptrón multicapa (MLP)** se concibe como un conjunto de unidades de procesamiento que transforman información mediante operaciones lineales y no lineales encadenadas. En sus primeros desarrollos, cada neurona realizaba una suma ponderada de sus entradas seguida de una función de activación, permitiendo a las RNAs captar patrones no lineales.

Este concepto resultó imprescindible al demostrarse que los perceptrones de una sola capa no podían resolver problemas de clasificación no lineal, como el famoso problema XOR [32].

De este modo, se introdujo la noción de **capas ocultas**, ubicadas entre la capa de entrada y la de salida, permitiendo a la red aproximar funciones arbitrarias siempre que contara con suficientes unidades y profundidad. Y esta palabra, “suficientes”, es el verdadero problema a la hora de llevar este método a la práctica, ya que no se conoce ningún resultado que pueda dar una idea acerca de cuántas de estas neuronas se necesitan para resolver un problema concreto.

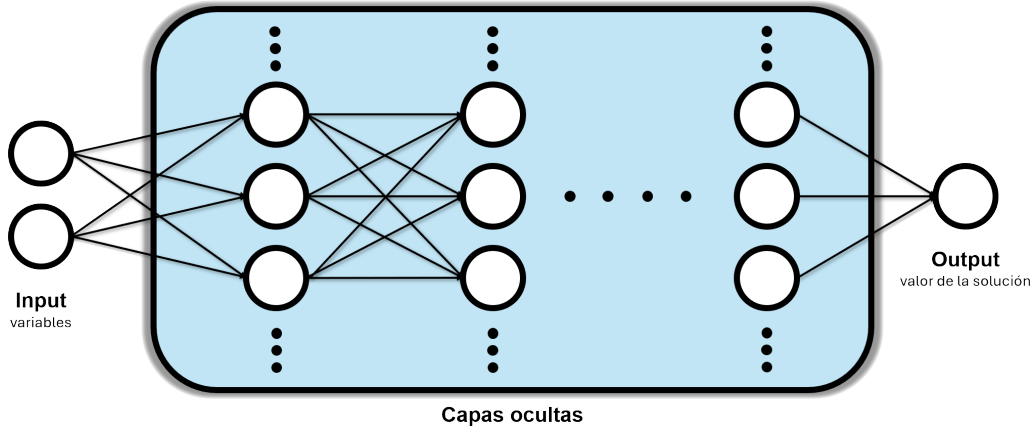


Figura 3: Diagrama genérico de los MLPs con capas completamente conectadas.

Dentro de las redes feedforward, el MLP se identifica como la subclase elemental. La capa de entrada recibe los valores de las características originales, las capas ocultas generan representaciones intermedias de mayor abstracción y la capa de salida emite la predicción final. Cada capa lineal se expresa mediante $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, con $\mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)})$ la **función de activación** y ℓ el índice de la capa, donde ϕ puede ser, por ejemplo, la famosa función *Rectified Linear Unit* (ReLU).

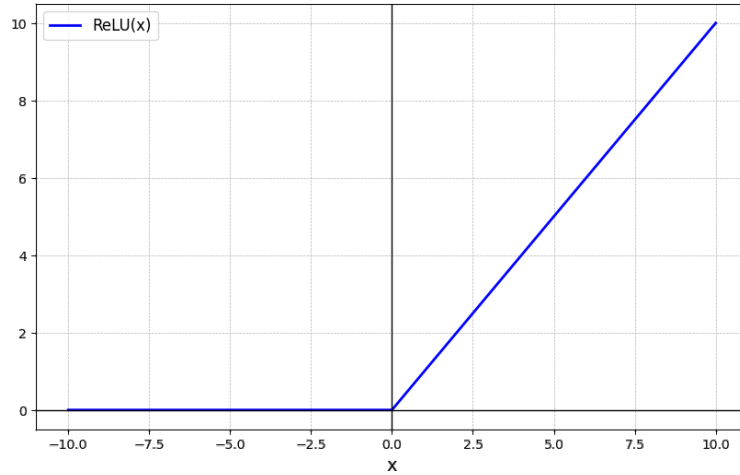


Figura 4: $\text{ReLU}(x)$, función de activación usada en este trabajo [15].

$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ contiene en cada fila los pesos que conectan todas las neuronas de la capa $\ell - 1$ con una neurona de la capa ℓ , mientras que $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$ es un vector cuyos componentes son los sesgos de cada neurona de la capa ℓ .

Desde la perspectiva de la aproximación de funciones, fue demostrado que las redes así estructuradas, con una capa oculta dotada de un número suficientemente grande de neuronas y empleando activaciones no lineales, gozaban de la *capacidad de aproximación universal*. Esta propiedad, establecida en los años ochenta, fundamenta la capacidad de estas redes para modelar relaciones complejas entre variables de entrada y salida.

Teorema (Aproximación Universal): Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ una función continua definida sobre un subconjunto compacto $K \subset \mathbb{R}^n$. Entonces, para todo $\varepsilon > 0$ existe una red neuronal de una sola capa oculta (es decir, un perceptrón multicapa con activación no lineal arbitraria) con un número finito de neuronas en la capa oculta tal que, al denotar su salida por $\hat{f}(x)$, se cumple

$$\sup_{x \in K} |\hat{f}(x) - f(x)| < \varepsilon.$$

Para minimizar la función de pérdida mencionada anteriormente, se hará uso del algoritmo de **retropropagación** del error, el cual consiste en actualizar todos los parámetros del modelo haciendo uso de un **optimizador** basado en descenso de gradiente.

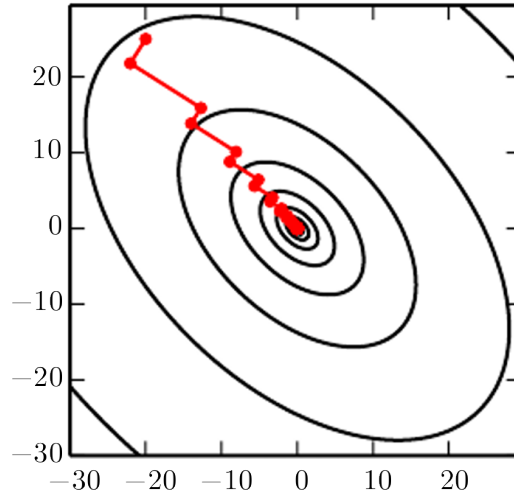


Figura 5: Diagrama visual del progreso del error hacia curvas de nivel de la función de pérdida de menor valor en el espacio de alta dimensión.

De esta forma, el entrenamiento de la red neuronal, sea MLP o las que veremos a continuación, consta de tres fases fundamentales con quizás algún cambio de notación.

1. Propagación hacia adelante: Se introducen las entradas $\mathbf{x}$ en la capa de entrada y se calcula sucesivamente la salida de cada capa lineal

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)}),$$

hasta obtener la predicción final $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$. A continuación se evalúa la función de pérdida $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$.

2. Retropropagación: Se calcula el gradiente $\nabla_{\boldsymbol{\theta}} \mathcal{L}$. Aplicando la regla de la cadena se obtiene, para cada peso y sesgo,

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}} \frac{\partial \mathbf{z}^{(\ell)}}{\partial W^{(\ell)}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}}.$$

3. Actualización de parámetros: Una vez disponibles los gradientes, el optimizador aplica la regla de descenso de gradiente

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t),$$

donde $\boldsymbol{\theta}$ agrupa todos los pesos $W^{(\ell)}$ y sesgos $\mathbf{b}^{(\ell)}$ y η es la **tasa de aprendizaje**.

Se ha observado que una tasa de aprendizaje demasiado alta induce oscilaciones e inestabilidad, mientras que una demasiado baja ralentiza el entrenamiento. Para ajustar este parámetro de forma dinámica, se incorporan **planificadores** o schedulers que pueden reducir gradualmente la tasa de aprendizaje de diversas maneras. Veremos en la implementación de los modelos como finalmente se eligió una reducción gradual de la tasa de aprendizaje basada en el estancamiento del error de validación.

2.1.3 Las redes neuronales convolucionales

Las **redes neuronales convolucionales** (CNN) procesan datos organizados en una cuadrícula bidimensional, como las imágenes, donde cada punto de la cuadrícula corresponde a un píxel con un valor de intensidad o vector de canales.

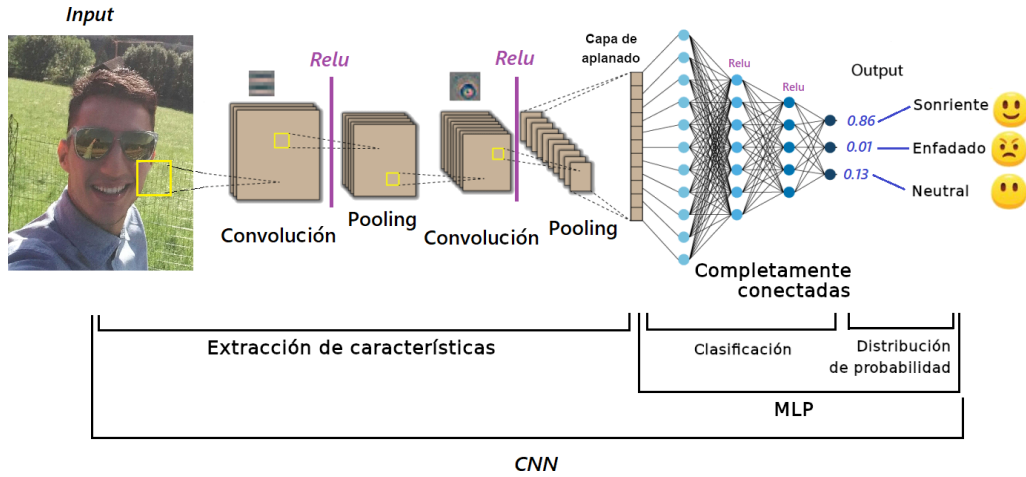


Figura 6: Esquema de la estructura CNN. La imagen de entrada se representa como una matriz de valores (intensidades o vectores RGB) organizados en filas y columnas, a los cuales se les aplica una serie de filtros para extraer características que se abstraen por un MLP en un espacio de menor dimensionalidad [7].

Estas redes se basan en el concepto de *conectividad local*, el cual nos viene a decir que cada neurona convolucional solo puede ver una pequeña ventana de esa cuadrícula, llamada **campo receptivo**, reduciendo enormemente el número de parámetros en comparación con una capa totalmente conectada como las del MLP.

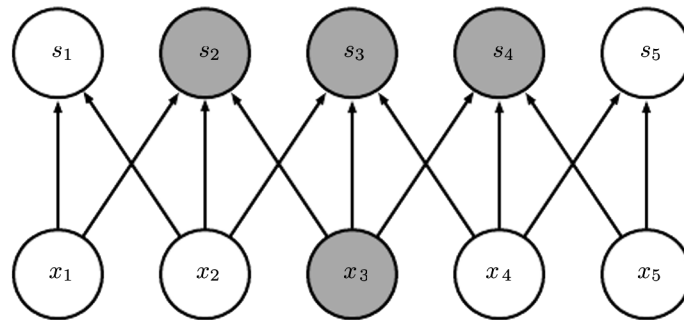


Figura 7: Cada unidad de la capa convolucional está conectada únicamente a un bloque de la capa previa.

Definiremos **kernel** o filtro como un pequeño tensor de parámetros entrenables que se utiliza para extraer características locales de una entrada. Intuitivamente, el kernel actúa como un detector local de patrones (bordes, texturas, formas) aprendiendo durante el entrenamiento a responder a las características más relevantes de los datos.

De manera más formal, un kernel de dimensiones $C_{\text{in}} \times k \times k$ es un conjunto de pesos

$$K = \{K_{c,u,v} \in \mathbb{R} \mid c = 1, \dots, C_{\text{in}}, \quad u, v = 0, \dots, k-1\}$$

y un *sesgo escalar* $b \in \mathbb{R}$, que se aplica a cada posición de la entrada X para producir la salida Y mediante la siguiente operación de convolución

$$Y_{i,j}^{(f)} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{c,u,v}^{(f)} X_{i+u,j+v}^{(c)} + b^{(f)}, \quad f = 1, \dots, C_{\text{out}}.$$

Así, cada canal de salida f es la respuesta global del filtro $K^{(f)}$ a lo largo de toda la entrada, respetando la estructura local y compartida. Aquí k se conoce como el **tamaño** del kernel, hiperparámetro que determina el campo receptivo local de cada neurona, C_{in} es el **número de canales de entrada** y C_{out} los de **salida**.

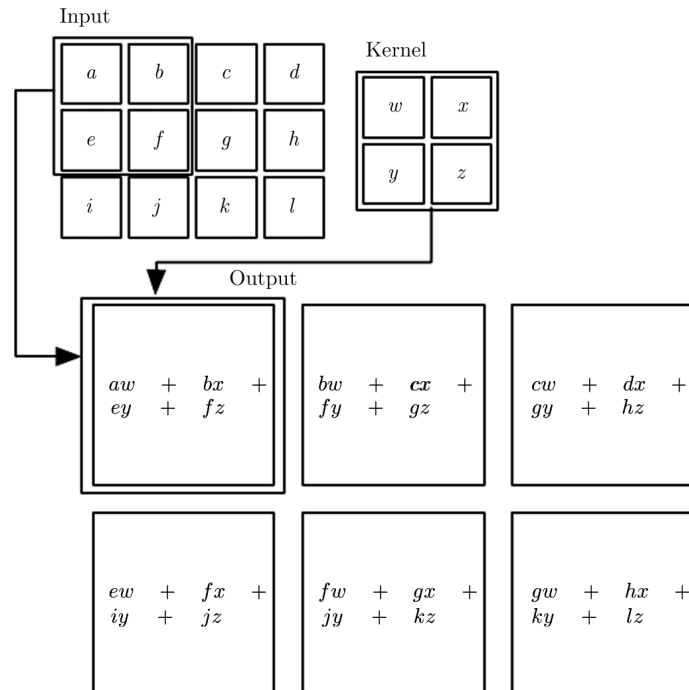


Figura 8: El mismo filtro K se aplica o *se desliza* por toda la imagen, generando un mapa de activación que indica la presencia de la característica asociada.

Esto nos lleva al concepto de *compartición de pesos*, pues un solo conjunto de parámetros K y b se reutiliza en todas las posiciones. Esto tiene dos ventajas fundamentales, la *invarianza espacial* (el detector responde a la misma característica, como un borde vertical, sin importar dónde aparezca) y la *reducción de parámetros* (en lugar de aprender pesos únicos para cada posición, sólo aprendemos k^2 por filtro, mejorando la eficiencia y consiguiendo que el modelo rinda mejor ante datos nunca vistos).

Existen más parámetros asociados al concepto de convolución discreta. Uno de ellos es el **stride** o deslizamiento, que controla el desplazamiento del filtro.

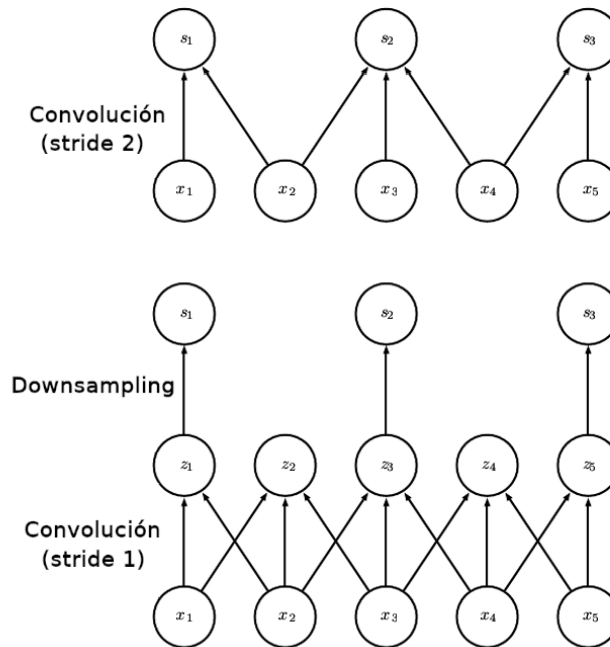


Figura 9: Aplicar una convolución de stride igual a 2 (arriba, un output por cada dos inputs) es equivalente a aplicar una convolución de stride igual a 1 y realizar el *downsampling* o reducción mostrado (abajo, un output por cada cada input).

También existe el **padding**, el cual se define como el número de elementos o píxeles que añadimos a la muestra tras la operación de convolución, alrededor de la neurona de salida. En concreto, trataremos el zero padding, que consiste en añadir concretamente elementos nulos. En la figura 10 se puede observar la utilidad del padding para controlar el tamaño de la red.

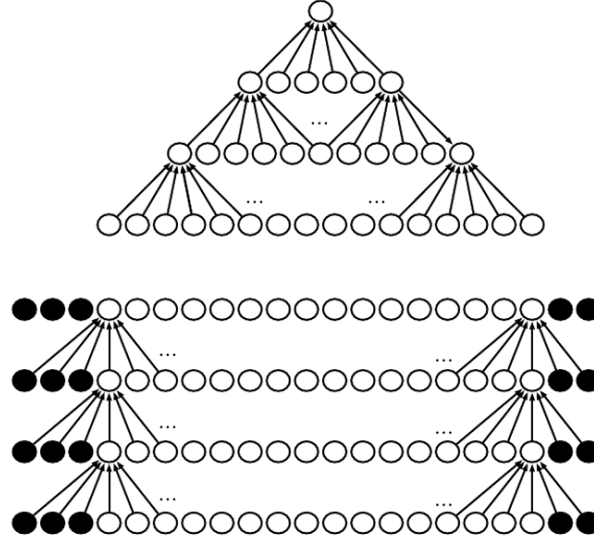


Figura 10: Detalle del padding. Vemos como añadiendo cinco ceros implícitos en cada capa, evitamos que la representación se reduzca con la profundidad de la red, permitiéndonos crear una red convolucional arbitrariamente profunda.

Por otra parte, el **pooling** es una operación de downsampling de la resolución espacial que se aplica sobre ventanas de tamaño $p \times p$ en cada mapa de activación. En el caso del *max-pooling*, que es el que utilizaremos y el más común, la salida Y en la posición (i, j) se define como

$$Y(i, j) = \max_{\substack{1 \leq u \leq p \\ 1 \leq v \leq p}} X(s i + u - 1, s j + v - 1),$$

donde s es el *stride* del pooling (definido como antes) y X es la entrada. Esta fórmula selecciona el valor más alto dentro de cada ventana, conservando la característica más fuerte y descartando información de menor intensidad.

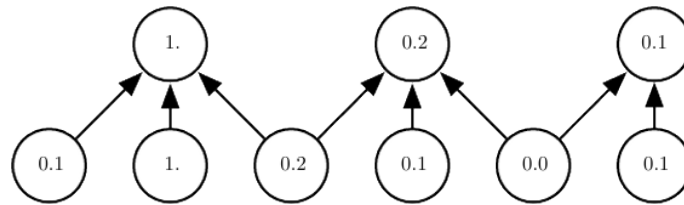


Figura 11: Ejemplo visual del max-pooling, para cada bloque se toma el valor máximo y se construye un mapa de salida de menor tamaño.

2.1.4 La arquitectura Transformer

Los **Transformers**, abreviados en este documento como **TRA**, son modelos basados en atención que han revolucionado el procesamiento de lenguaje natural. En lugar de procesar palabras una a una, representan cada token (unidad mínima de texto a procesar) como un punto en un espacio continuo mediante embeddings. La explicación general que daremos y su implementación para nuestro problema está muy ligada al artículo [3, “*Attention Is All You Need*”].

Un **embedding**, en procesamiento de lenguaje natural, es simplemente una forma de convertir palabras o fragmentos de texto en vectores numéricos, de modo que el modelo pueda entenderlos y operarlos matemáticamente. De esta forma, en lugar de tratar cada palabra como un identificador discreto (un número entero), asignamos a cada **token** (unidad mínima a representar, palabras en este caso) un punto en un espacio continuo de alta dimensión, de forma que palabras con significados parecidos, como “Francia” y “España”, terminan muy cerca en ese espacio, mientras que palabras sin relación quedan lejos unas de otras.

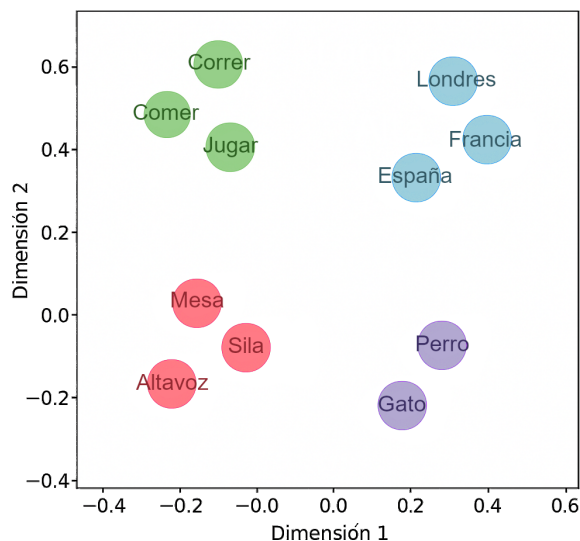


Figura 12: Visualización del concepto de embedding en el contexto del lenguaje natural.

Los Transformers deciden qué partes del texto son relevantes usando un sistema parecido a una conversación entre preguntas y respuestas internas, al cual llamaremos **atención**.

Se parte de una lista de palabras de las cuales se quiere saber cuánto hay que “atender” a cada una al procesarla. Para ello, el modelo crea tres parámetros o propiedades para

cada palabra, los cuales reciben los nombres de Q (query), K (key) y V (values) en la bibliografía.

Cada consulta se compara con todas las claves para ver qué tanta “afinidad” tienen. Cuanta más afinidad, más atención le damos al valor correspondiente. Así, al procesar “come”, el modelo podría fijarse mucho en el valor de “gato” y en el de “pescado”, y menos en las palabras “El” o “un”.

Este mecanismo de atención es el que nos permite conceptualmente lograr que el modelo relacione características distantes del espectro, como reconocer no solo que los picos en intensidad son interesantes sino también aprender a reconocer que el lugar en el que se ubican del espectro y la distancia relativa entre picos es interesante.

Para enriquecer aún más este fenómeno, el Transformer puede lanzar varias de estas atenciones al mismo tiempo. A cada una de estas instancias se le llama **cabeza de atención**, de forma que se podría interpretar que cada una de estas cabezas es un interlocutor en una conversación que postula sus propias consultas y claves y busca diferentes tipos de relaciones. Seguidamente, el modelo agrupa toda esta información en una única representación más completa.

En la figura 13 se puede ver de manera condensada este concepto, donde h sería el número de estas instancias y en la caja morada podemos apreciar el nombre que le da el artículo a este proceso de atención.

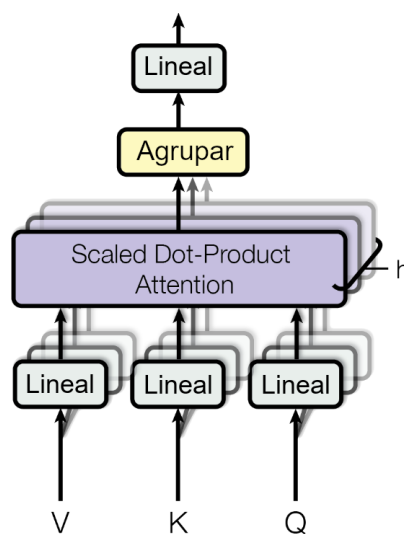


Figura 13: Mecanismo de atención con múltiples cabezas en paralelo.

Como la arquitectura no recorre la secuencia en orden, se requiere que añadamos **codificaciones posicionales** (PE) basadas en senos y cosenos

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad \text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right),$$

que se suman a los embeddings para distinguir si una palabra aparece al principio, en medio o al final de la oración, donde *pos* es la posición de la palabra.

Una vez aplicadas las atenciones, cada posición se refina de forma independiente con una red feedforward de dos capas lineales y activación ReLU.

En el campo del lenguaje, este conjunto de mecanismos permite traducir oraciones, responder preguntas o generar texto coherente. A nivel de entrenamiento no es distinto, desde el punto de vista de concepto, a los ya tratados. El ejemplo prototípico que se pone para entender la arquitectura y es el de *Chat Bot*, como **ChatGPT** o **Deepseek**, los cuales se basaron en sistemas parecidos para ofrecer al usuario conversaciones coherentes.

Igualmente, lo podemos utilizar para lo que lo queramos, pues al fin y al cabo nuestros datos, que serán los datos del espectro de un objeto estelar, son interpretables de igual forma como secuencias, las cuales pueden abstraerse a un espacio de alta dimensión.

Esta idea nace de que, al igual que hemos presentado la teoría en este trabajo, inicialmente se comenzó con redes MLP y CNN, las cuales se centran mayormente en extraer características más bien locales. Como veremos a continuación, para calcular consistente el redshift cosmológico se necesita no solo saber que pasa alrededor de cada punto del espectro, sino también entender dónde se encuentra esta y cómo se relaciona con el resto.

2.1.5 Regularización

Cuando entrenamos un modelo, buscamos que aprenda patrones generales de los datos que le permitamos ver, pero sin memorizar cada detalle específico ni pecar de ser demasiado superficial. El término **overfitting** o sobreajuste describe el caso en que el modelo aprende tan bien el conjunto de entrenamiento que ajusta excesivamente sus parámetros a las particularidades y el ruido del conjunto de entrenamiento. Como consecuencia, su capacidad de generalizar a datos nuevos disminuye drásticamente.

Por el contrario, el **underfitting** o subajuste ocurre cuando el modelo es tan sencillo o los datos tan pobres que ni siquiera llega a capturar las tendencias principales de los datos de entrenamiento, resultando en errores altos tanto en entrenamiento como en prueba independientemente del tiempo de entrenamiento.

Imaginemos, por ejemplo, que queremos entrenar una red CNN con un millón de imágenes de la misma taza de café para que nos diga si una foto es o no una taza de café. Seguramente, si le pasamos una foto de la taza usada durante el entrenamiento, nos dirá que es una taza de café, pero si le pasamos una taza de, por ejemplo, otro color, muy posiblemente no la reconozca. Esto ocurre porque la red no ha sido capaz de abstraer el concepto taza a partir de los datos suministrados.

Las técnicas para combatir el underfitting no van más allá de intentar construir un buen dataset y ampliar o cambiar la estructura del modelo, ambas bastante cuidadas en este trabajo. El estudio del overfitting va mucho más allá, pues cuando ya tenemos un modelo lo suficientemente grande y complejo, que es capaz de abstraer muchos patrones de todos los datos, aunque no sean para nada relevantes, se vuelve bastante difícil ir obteniendo mejores resultados en datos ajenos al entrenamiento. En general, utilizar capas convolucionales suele ayudar en este sentido, como ya mencionamos anteriormente.

Por tanto, la dificultad de este campo muchas veces radica en encontrar un equilibrio en la complejidad del modelo y en el tratamiento de este problema. La técnica más nombrada, de la cual haremos uso en este trabajo, es el **dropout** [12], el cual consiste en apagar aleatoriamente una fracción p de las neuronas de una capa, generalmente fijando su salida a 0. Se suele utilizar generalmente entre capas lineales y se realiza siempre en la fase de feedforward del entrenamiento, de forma que al final de cada época todas las neuronas del modelo tienen importancia.

También utilizaremos un hiperparámetro llamado **decaimiento de pesos** o weight decay, que actúa también como regularizador penalizando la magnitud de los pesos y mejorando la capacidad de generalización del modelo [28].

2.2 El redshift en cosmología

2.2.1 Definición y conceptualización

Antes de definir el redshift e incrustarlo en el marco de la cosmología, vamos a fijar la notación y los conceptos a utilizar. La mayoría de ellos son divulgativos y se presentan para poder resaltar la importancia de este parámetro.

Se le llama *tiempo cósmico* (t) a la coordenada temporal común a todos los observadores en reposo, independientemente de cómo se expanda el Universo, para tener un marco común de tiempos. Además, llamaremos t_E al instante en el que se emitió luz por un objeto y t_R al instante en que se recibió.

Definimos el factor de escala $R(t)$ como una función del tiempo cósmico t que cuantifica el tamaño relativo del espacio. Un incremento de $R(t)$ indica expansión, un decremento, contracción. Normalmente se escoge $R(t_0) = 1$ para el tiempo presente.

Se hablará de la velocidad de recesión v_R como la rapidez a la que una galaxia se aleja por la expansión cósmica.

La *distancia comóvil* L es la separación constante entre dos puntos en coordenadas que se expanden junto con el universo. En otras palabras, mide cuán lejos estarían dos objetos si congeláramos la expansión cósmica.

Ahora sí, el **redshift** z se define como el desplazamiento relativo de la longitud de onda de una línea espectral entre su valor observado λ_{obs} y su valor propio (o en reposo) λ_0

$$z = \frac{\lambda_{\text{obs}} - \lambda_0}{\lambda_0}, \quad (2.1)$$

y puede expresarse en términos del factor de escala del Universo mediante la conocida relación

$$1 + z = \frac{R(t_R)}{R(t_E)}, \quad (2.2)$$

pues este efecto no es más que el resultado de que el Universo se haya expandido durante todo el trayecto que ha recorrido la luz, expandiéndose el propio espacio que define el periodo de la onda.

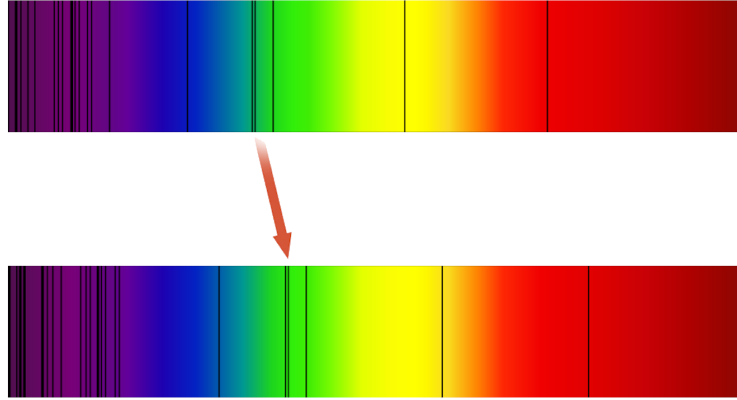


Figura 14: Cuánto más alejada está la fuente, mayor redshift presenta su espectro debido a la expansión del Universo.

Cuando z es muy pequeño ($z \ll 1$), el redshift puede interpretarse como un efecto Doppler no relativista, de manera que la relación (2.1) se asocia directamente con la velocidad de recesión v de la fuente respecto al observador, dando la aproximación clásica

$$z \approx \frac{v}{c}, \quad (2.3)$$

válida cuando $v \ll c$.

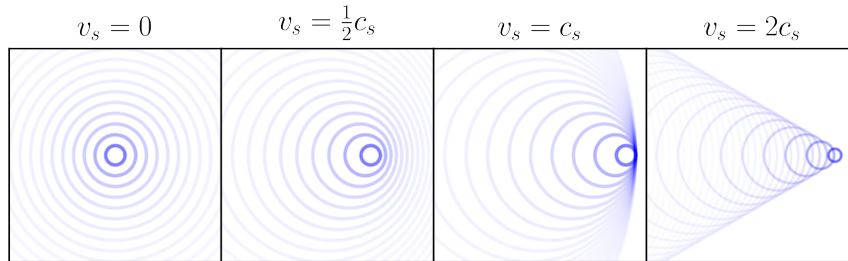


Figura 15: Sea c_s la velocidad del sonido, el efecto Doppler se magnifica cuando un objeto que emite sonido se mueve hacia la derecha con una velocidad cada vez mayor en comparación con c_s [5]. En esta figura el modelo es acústico, pero lo mismo le ocurre a un objeto luminoso cuya velocidad de recesión se acerca a la velocidad de la luz.

La **ley de Hubble** establece que la velocidad de recesión de una galaxia es proporcional a su distancia L respecto al observador

$$v = H(t)L,$$

donde $H(t)$ es la tasa de expansión del universo en el tiempo t . Para la época presente t_R , $H(t_R)$ se denomina constante de Hubble. Combinando esta ley con la aproximación

Doppler, se obtiene la relación lineal entre redshift y distancia, válida para $z \ll 1$

$$z \approx \frac{H(t_R) L}{c}. \quad (2.4)$$

Destacar que en el redshift final con el que trabajamos aquí, que es esencialmente el que se mide, es la combinación de ciertos fenómenos que le ocurren a la luz además de la expansión del Universo, como lo pueden ser el redshift gravitacional [8] o las lentes gravitacionales [9], que pueden inducir algo de ruido a la hora de inferir parámetros como la constante de Hubble presentada anteriormente.

2.2.2 Importancia del redshift

El redshift es una de las herramientas más poderosas de la cosmología moderna, puesto que constituye un indicador directo de cuánto se ha estirado la longitud de onda de la radiación electromagnética emitida por un objeto astronómico desde el instante de emisión hasta el de recepción. En la sección previa definimos esta magnitud en la ecuación (2.1) y mostramos su vinculación con el factor de escala cósmico en la ecuación (2.2).

A partir de estas relaciones fundamentales, podemos extraer información crucial sobre la velocidad de recesión de galaxias y cuásares (2.3), así como estimar las distancias comóviles y luminosas de los objetos más lejanos. Tales mediciones permiten, a su vez, inferir los parámetros que rigen la dinámica y geometría global del universo.

Utilizando la Teoría de la Relatividad, el redshift marca cuánto se ha dilatado la longitud de onda y nos permite encontrar propiedades profundas sobre la trayectoria de un fotón. Esto nos ha permitido descubrir conceptos como el de materia oscura y energía oscura.

El redshift nos sirve para calibrar la constante de Hubble antes mencionada (2.4), de forma que mediante algunos tipos supernovas (faros estándar), hemos alcanzado una precisión cercana al 1 % en H_0 . Sin embargo, la comparación con los valores inferidos de la radiación de fondo cosmológica emitida a edades tempranas del Universo revela la llamada “tensión de Hubble”.

Esta discrepancia podría llevarnos a una nueva física más allá de la relatividad general

o a pequeños sesgos en la determinación de z , pues cada décima de error en el redshift repercute en kilómetros por segundo por megapársec y, por tanto, en nuestras estimaciones.

En definitiva, el redshift es el nexo que vincula la teoría de la relatividad general con la observación astronómica y la inferencia de los parámetros cosmológicos. Gracias a él, podemos reconstruir la historia de la expansión, medir con precisión distancias de miles de millones de años luz y calibrar las constantes que definen el destino de nuestro universo.

2.2.3 Técnicas de medición

La historia comienza con Vesto Slipher, que durante su trabajo en el Lowell Observatory recibió el encargo de estudiar el espectro de la “nebulosa de Andrómeda”. Tras meses de pruebas encontró que, cambiando a un prisma de gran dispersión angular, abriendo la rendija del colimador y usando una lente de cámara de foco muy corto, podía obtener **espectrogramas** suficientemente nítidos pese a la modesta apertura del telescopio. Así logró obtener el primer espectro claro de Andrómeda y medir que sus líneas estaban desplazadas *hacia el azul* (blueshift), lo que indicaba una velocidad de aproximación de unos 300 km/s. En los años siguientes midió velocidades radiales de otras 14 *nebulosas espirales*, hallando que la mayoría estaban desplazadas al rojo, es decir, se alejaban de nosotros [4].

Este fue el inicio de la **espectroscopía** en astronomía, ya que permitía estudiar objetos luminosos cada vez más débiles sin necesidad de instrumentos colosales, simplemente refinando esta idea de difractar la luz. El proceso a seguir sigue siendo el mismo que se lleva a cabo hoy en día, solo que ahora es mucho más sofisticado y automático.

Se comienza recolectando la luz proveniente de un objeto (galaxia, nebulosa, Cefeidas, ...). Seguidamente esta entra en un espectrógrafo (inicialmente un prismático, más adelante redes de difracción) que separan la luz en sus distintas longitudes de onda, de manera que utilizando placas sensibles que se exponen al espectro durante horas o días (para conseguir una relación señal/ruido adecuada). Finalmente, se calibran los espectros a partir de lámparas (sobre todo se comparan con los espectros de emisión producidos por lámparas de H, Hg, Ne, Ar) para identificar los patrones de absorción o emisión, pudiendo calcular finalmente z con la relación ya vista.

Un objeto astronómico, como una estrella, emite luz a lo largo de todo el espectro, un continuo. Cuando la luz de una estrella atraviesa un medio (gas de una nebulosa, la propia atmósfera de la estrella, ...) ciertas longitudes de onda son absorbidas por los elementos allí presentes, produciendo líneas oscuras sobre el continuo que forman el patrón antes mencionado (espectro de absorción).

La energía absorbida por el gas se reemite luego en todas las direcciones, también en los colores específicos característicos de los elementos presentes, produciendo líneas brillantes a ciertas longitudes de onda; esto se conoce como espectro de emisión.

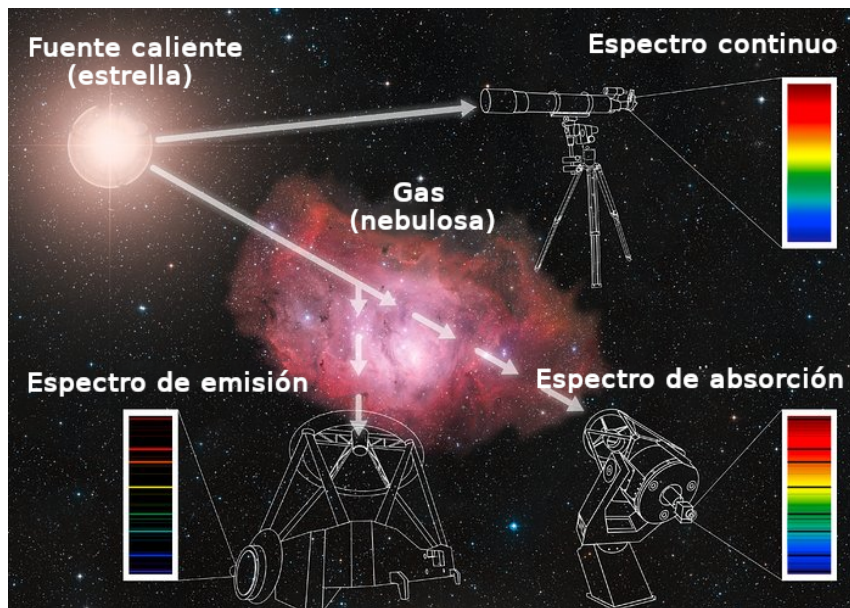


Figura 16: La espectroscopía es clave para poder medir con precisión estos espectros, pues al estar tratando con longitudes de nanómetros se necesita tener un pixelado ridículamente pequeño para poder distinguir correctamente unas líneas de otras.

Para las definiciones formales y algunas ecuaciones presentadas en este apartado nos hemos basado en el libro [2, capítulo 6].

Capítulo 3

El Conjunto de Datos

3.1 Recolección y preparación de datos

3.1.1 Fuente principal

Todo el grueso de este proyecto se basa en datos suministrados por la **Sloan Digital Sky Survey**, abreviada como **SDSS** [16]. Este inmenso proyecto de mapeo astronómico combina técnicas de imagen multibanda y espectroscopía de fibras para generar catálogos de más de 1000 millones de objetos y 4 millones de espectros, con un alcance que abarca desde estudios de la estructura a gran escala del Universo hasta la física estelar en nuestra propia galaxia.

Su propósito fundamental es proporcionar un conjunto de datos homogéneo y de gran volumen que permita estudiar la distribución de la materia en el Universo a múltiples escalas. El sondeo fotométrico estaba diseñado para detectar galaxias, cuásares y estrellas hasta los límites de magnitud establecidos, facilitando estudios estadísticos de correlación angular y fotométrica en grandes volúmenes cósmicos.

Mediante el sondeo espectroscópico, SDSS ha medido redshifts de millones de galaxias y cuásares, lo que ha permitido trazar la estructura tridimensional del Universo y detectar la escala característica de las *oscilaciones acústicas de bariones* (BAO), clave para determinar la *historia de expansión cósmica*, todo esto de manera transparente y accesible por todo el mundo.

El telescopio principal que utilizan ofrece excelente calidad de imagen en un campo de visión bastante amplio y con distorsiones geométricas muy pequeñas en el rango de longitudes de onda 3000–10600 Å, el cual va a ser el rango en el que se muevan los espectros con los que vamos a trabajar [17].

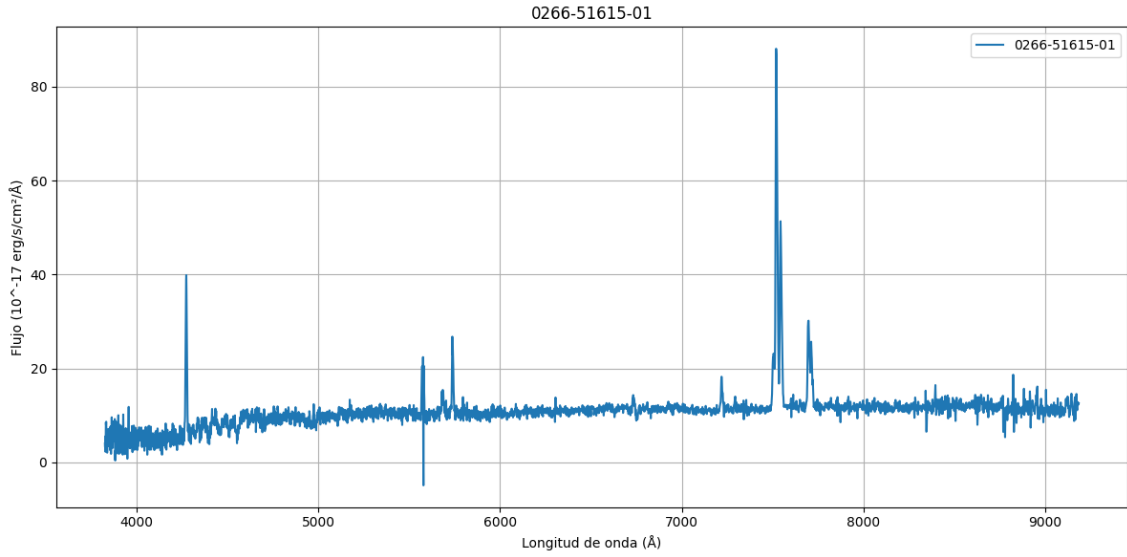


Figura 17: Espectro de la galaxia 0266-51615-01 de la SDSS.

En modo espectroscópico, el telescopio funciona de forma tradicional, enfocando luz en placas de aluminio perforadas, cada perforación alineada con un objetivo y conectada a una fibra óptica. A partir de esto, un *pipeline espectroscópico* (conjunto estructurado de algoritmos, scripts y procedimientos automatizados) procesa las mediciones para convertirlas en datos calibrados y útiles de longitud de onda y flujo.

La cámara utilizada integra cinco filtros diseñados para maximizar la cobertura espectral y minimizar contaminaciones.

- **u (355 nm)**: evita el salto de Balmer.
- **g (468 nm)** y **r (616 nm)**: bandas anchas que evitan la línea de cielo de 5577 Å.
- **i (748 nm)**: corta antes de bandas de *OH* atmosférico.
- **z (893 nm)**: evita absorciones por agua.

En la base de datos, estos filtros tienen asociado un valor conocido como la magnitud del filtro, formando un vector $\mathbf{x} = (m_u, m_g, m_r, m_i, m_z)$ que puede ser interesante para estimar el redshift según la idea presentada por [10] mediante polinomios de alto orden optimizados.

Así, se consigue un valor único de z , acompañado de

- una incertidumbre **zErr** derivada de la anchura del pico de correlación y de la covarianza de los ajustes de línea
- un parámetro **zWarning** [23] que alerta sobre posibles problemas (mucho ruido, líneas superpuestas, pocos picos fiables)

lo cual nos garantiza cierto nivel de confiabilidad.

3.1.2 Estructura de los archivos

Para obtener los datos utilizaremos los dos servidores en los que se alojan. El *Catalog Archive Server* o CAS [20], basado en **SQL** Server [31], nos suministra todos los datos que necesitamos de un cierto objeto a través de la tecnología SkyServer y CasJobs, dándonos además la referencia para descargar los datos brutos de todos los objetos de la base de datos mediante el *Science Archive Server* o SAS [21]. La tecnología **SQL** es la más usada en el campo de las bases de datos y funciona mediante consultas más intuitivas y amigables que los lenguajes de programación habituales, de las comentaremos y mostraremos ejemplos más adelante.

El procesamiento de los espectros mencionado anteriormente se realiza mediante el pipeline *spec2d* [24], que toma las exposiciones bidimensionales en bruto y realiza una serie de correcciones que desembocan en la producción de espectros unidimensionales calibrados en flujo y longitud de onda, junto con sus estimaciones de ruido e información de los filtros aplicados para poder revertirlos. Múltiples exposiciones del mismo campo se apilan para mejorar la relación señal-ruido, generándose archivos **.fits** que contienen el espectro final, la varianza y los datos de calibración para cada fibra.

El formato **FITS** (Flexible Image Transport System) [22] organiza la información en bloques que combinan un encabezado con datos. El encabezado contiene la información esencial, como el tipo de extensión (por ejemplo, imagen o tabla), el nombre del objeto observado y la fecha de la observación. A continuación, los datos pueden consistir en una imagen de píxeles o en una tabla con valores numéricos (por ejemplo, un espectro).

En la SDSS, el bloque principal incluye datos generales de la observación y los bloques

siguientes alojan, de manera ordenada, los espectros unidimensionales, su varianza y cualquier información adicional de calibración o calidad.

En este caso, un archivo `.fits` típico incluye

- HDU[0]: datos generales de la observación.
- HDU[1]: vectores esenciales, como **flux** (flujo) y **loglam** (logaritmo de la longitud de onda).
- HDU[2]: tabla de propiedades del objeto, que contiene el identificador **objid** único y su **redshift** medido, junto a multitud de metadatos de clasificación.
- HDU[3]: parámetros de las líneas espectrales detectadas vinculadas al objeto y utilizadas para refinar el redshift y el análisis de emisión, entre otras cosas.

3.1.3 Otras fuentes

El **NASA/IPAC Extragalactic Database (NED)** [25] es un repositorio centralizado operado por el *Infrared Processing and Analysis Center* (IPAC) de Caltech y financiado por la NASA. Su objetivo principal es recopilar y organizar información de objetos extragalácticos (galaxias, cuásares, fuentes de radio, rayos X e infrarrojas, entre otros) provenientes de múltiples catálogos y publicaciones especializadas. En este sentido, NED funciona como un catálogo maestro de identificadores y posiciones, ofreciendo datos básicos como coordenadas, corrimientos al rojo (redshifts) y referencias bibliográficas asociadas a cada objeto estudiado.

La base de datos del NED también se implementa sobre un motor relacional **SQL**, de forma que mediante un procedimiento similar al anterior hemos sido capaces de obtener alrededor de 90,000 archivos con espectros astronómicos [26].

3.2 Preprocesamiento de los datos

En este apartado vamos a exponer el camino que han recorrido los datos desde que fueron descargados hasta que han podido ser utilizados para entrenar y evaluar nuestras redes neuronales.

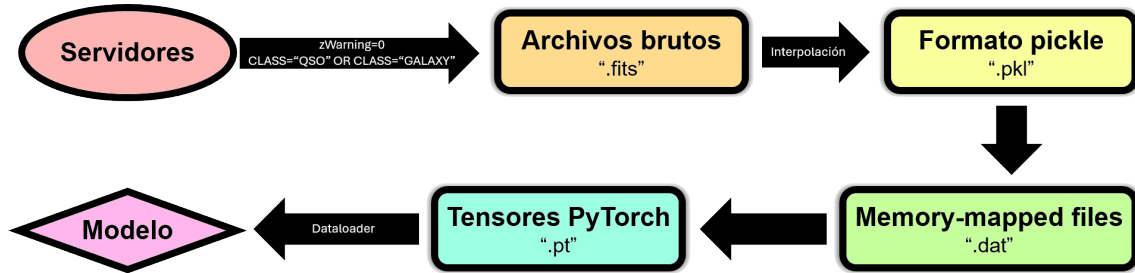


Figura 20: Esquema de la obtención y preprocesamiento de los datos.

Creemos que este análisis es interesante presentarlo debido al gran volumen de los mismos, el cual nos ha llevado a ir descubriendo nuevos formatos y técnicas para poder seguir adelante en nuestra tarea. En muchas ocasiones se ha de trabajar con muchos datos, de forma que no hay que despreciar la utilidad de este proceso.

3.2.1 Descarga de los archivos brutos

Comenzamos la parte práctica de este trabajo construyendo de manera iterativa una consulta SQL. El lenguaje SQL no es más que una manera de comunicarse con el servidor CAS enviando un mensaje con una serie de condiciones, como por ejemplo “quiero que ordenes la base de datos en función del redshift z de menor a mayor y me des las 100 primeras entradas”.

Este mensaje, un poco más refinado, ha sido el que nos ha ido permitiendo descargar la base de datos completa, pues al ser una base de datos tan grande hay que proceder por bloques para no saturar al servidor. De esta forma, descargamos unos pocos datos, de entre todos ellos extraemos el que tiene mayor z y este redshift es el que usamos como cota inferior para la siguiente consulta.

El procedimiento anterior nos devuelve una clave por objeto astronómico que podemos utilizar para realizar una consulta en el servido **SAS**, el cual nos dará el archivo **.fits** correspondiente a cada clave. La implementación de descargas en paralelo en esta parte nos ha permitido reducir un tiempo de descargas de semanas a un par de días, lo cual debe ser tenido también en cuenta.

Comentar que los modelos han sido entrenados únicamente con espectros de Galaxias y **QSO** (*Quasi-Stellar Object*, cuásares $\equiv$ galaxias jóvenes u objetos muy luminosos en el centro de estas) únicamente con el parámetro antes mencionado **zWarning** = 0 que nos garantiza cierta calidad.

3.2.2 Manipulación de los datos

Manejo dimensional y uniformidad

El paso posterior que seguimos fue el de crear un bucle que, a partir de todos los **.fits** descargados en la sección anterior, va procediendo de igual forma por lotes identificados por un sufijo, como se puede ver en la carpeta de [resultados](#).

Como conocemos la estructura de estos archivos, comenzamos extrayendo el flujo (**flux**), la longitud de onda (**wave/wavelength**) y también el valor de redshift **z**. A cada espectro se le aplica la función de interpolación para homogeneizar el tamaño de cada muestra, insertando nuevos puntos intermedios usando la media aritmética

$$\lambda_{\text{nuevo}} = \frac{\lambda_i + \lambda_{i+1}}{2}, \quad f_{\text{nuevo}} = \frac{f_i + f_{i+1}}{2},$$

hasta alcanzar los 5000 valores de flujo y longitud de onda. Estos puntos se añaden convenientemente en zonas en las que el flujo varía rápidamente, para acentuar la importancia de los picos de cara al entrenamiento, pues ya que tenemos que unificar obligatoriamente la dimensión de entrada al menos aprovechamos el proceso.

El resultado se almacena en un diccionario con todas las muestras del lote y tres claves por muestra, **wavelength**, **flux** y **redshift**, en formato pickle **.pkl**. Finalmente, se eliminan los archivos **.fits** ya procesados para verificar que se han procesado todos.

El formato anterior se utilizó en gran parte del trabajo para entrenar los modelos y obtener los primeros resultados, pero a medida que se pretendía evaluar nuestras RNAs en conjuntos más grandes (para hacernos una idea, la base de datos completa maneja más de 3 millones de espectros, cada uno ahora con 10,001 dato) hacía que cada optimización que se le pudiese hacer a la carga de datos se notase enormemente.

Hay dos factores clave a la hora de preparar los datos para evaluar, el primero es cargarlos en la memoria y el segundo es tensorizarlos (convertirlos a un formato que la red neuronal entienda). A continuación mostraremos el proceso de conversión a memory-mapped files `.dat`, que aliviaban bastante el primer problema y nos permitieron ir definiendo de una manera más manejable los distintos conjuntos de datos o datasets, permitiéndonos hacer las primeras pruebas en conjuntos grandes necesarias para evaluar el poder de regresión o generalización de la red en datos no vistos.

El procedimiento se basa esencialmente en ir cargando los archivos `.pkl` de los que disponemos ahora mismo, reservamos la memoria destinada para cada uno de los tres canales. A continuación, de manera iterativa, vamos escribiendo los datos de todas las muestras, resultando en tres archivos distintos como podemos ver en la carpeta de [datos](#). Este tipo de archivos tienen la ventaja de que podemos precargarlos sin que ocupen memoria y podemos acceder poco a las muestras que queramos, siendo mucho más manejables.

Entendiendo que para este punto ya tenemos todos los archivos de la base de datos dispuestos en la base de datos, describiremos a continuación como tensorizarlos y agruparlos en el formato con el que ya trabajaremos hasta el final.

Construcción del dataset de entrenamiento

Este es el apartado más interesante, pues en el se describe como se han elegido las muestras utilizadas para entrenar los modelos. Una cosa que tiene que quedar clara y es muy importante es que, pese a no tener datos de estrellas en este conjunto de datos (eliminando unos 300,000 objetos de redshifts muy bajos), la distribución del redshift de las muestras medidas es extremadamente poco uniforme.

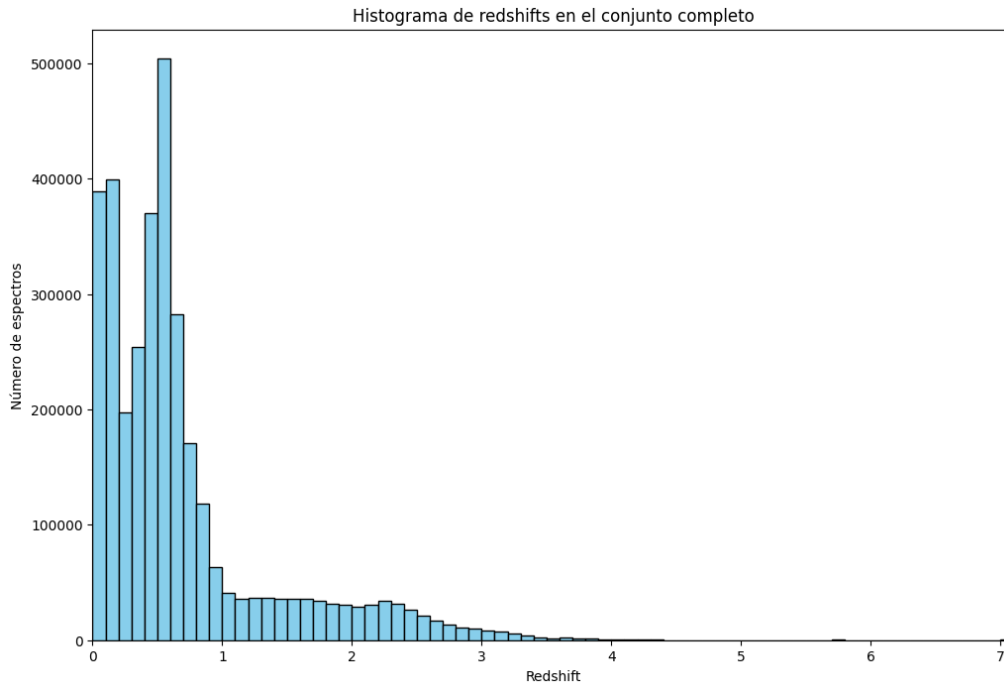


Figura 21: Distribución del número de muestras en función de su redshift, 3,372,890 espectros.

Esto nos lleva a querer mantener un equilibrio al construir el dataset de entrenamiento. Un dataset demasiado grande produciría sobreajuste, mientras que un dataset demasiado pequeño no nos bastaría para poder hacer ver al modelo un número representativo de muestras en cada rango de redshift. Si tomamos muestras aleatorias de todo el conjunto, prácticamente no tendríamos datos con $\text{redshift} \geq 4$.

Se decidió elegir pues un dataset de **entrenamiento** de tamaño 504,724, el 15 % del tamaño total del dataset. Este número lo repartimos de manera balanceada por rangos, $(0, 0.1)$, $(0.1, 1)$, $(1, 2)$, $(2, 3)$, $(3, 4)$ y $(4, 8)$, donde a partir de $z \geq 4$ tomamos todas las muestras. La esperanza de que el modelo no desarrolle sobreajuste para redshifts altos se basa en que este rango no contribuye prácticamente al error ($\approx 0.9\%$), sin dejar de ser positivo que colabore lo máximo posible a él.

Así pues, construimos un bloque de código que nos permita completar aleatoriamente estos rangos hasta llegar al valor requerido. Seguidamente, estos se utilizan para ajustar un **normalizador** (scaler) que se utilizará siempre para unificar (normalizar en media y varianza) los datos de longitud de onda y flujo antes de pasárselos al modelo, sea para entrenar o para evaluar. Seguidamente, los datos ya normalizados se tensorizan, convirtiéndolos a formato tensor `.pt` de la librería `PyTorch` de `Python`, la cual es la que contiene también todo lo necesario para crear los modelos.

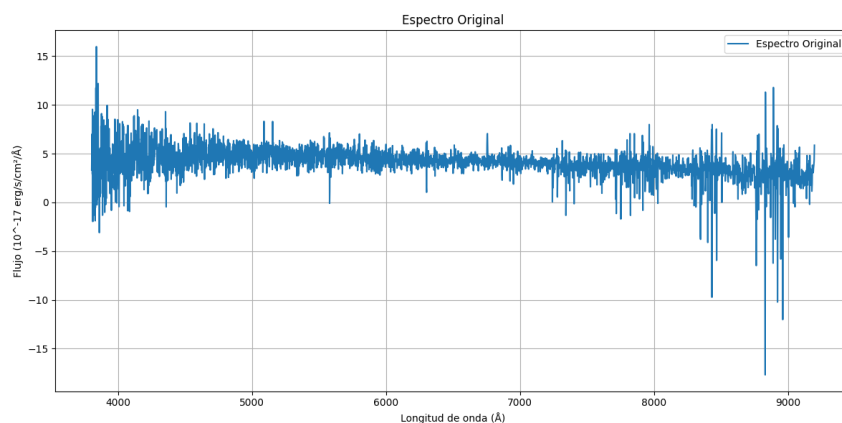


Figura 22: Datos sin normalizar.

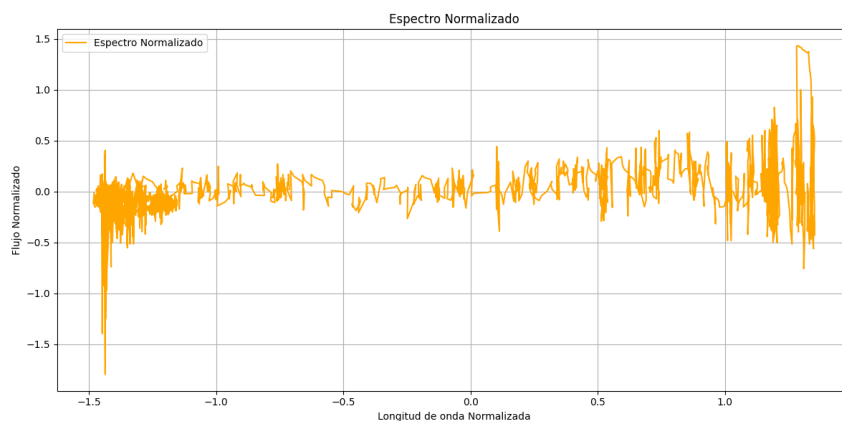


Figura 23: Muestra normalizada con el scaler.

Duplicaremos el dataset de **entrenamiento** (train) aquí creado, asignándole a uno el sufijo `_600k` y a otro `_1M`. Esto cobrará más sentido a continuación, pues para la creación del dataset de **validación** (test) comenzamos haciendo el mismo proceso, pero esta vez llegando a un tamaño total de 95,276 espectros, excluyendo los ya seleccionados anteriormente. Este dataset también llevará el sufijo `_600k`, de forma que junto al dataset de entrenamiento anterior se conocerán como el dataset `_600k`.

Seguidamente, realizamos una ampliación de este dataset de validación a 595,276 muestras para probar, en este caso, si ayuda a la regresión de los datos con $z \geq 4$. Las muestras con las que se amplían también se eligen aleatoriamente en los rangos anteriores pero permitiendo en este caso datos que estén en el dataset de entrenamiento, incluyendo de esta forma todas las muestras con $z \geq 4$. Notar que para $z \leq 4$ la aleatoriedad ya nos garantiza que la mayoría de los datos van a ser nuevos por la proporción, pues estamos hablando de un 85 % ajenos al dataset de entrenamiento frente a un 15 % que si están. Estos dos datasets, entrenamiento y validación formarán pues el que se conocerá como dataset `_1M`.

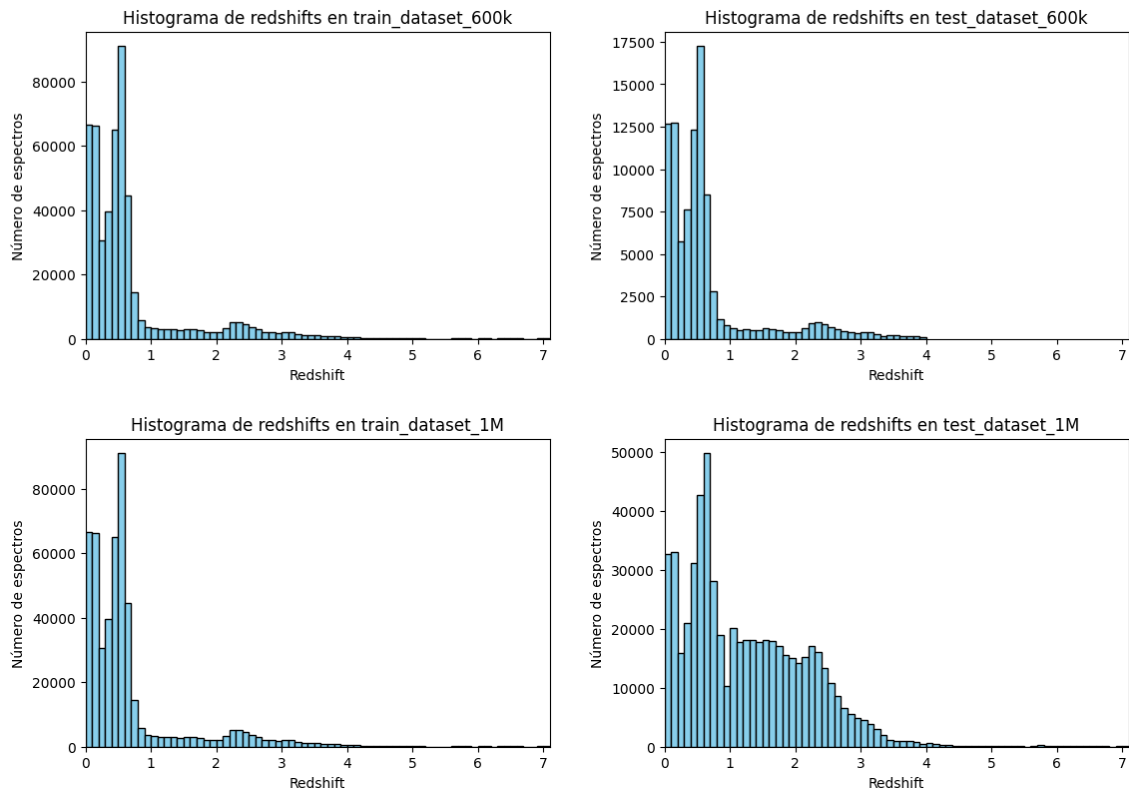


Figura 24: Histogramas que muestran la distribución del redshift en los distintos datasets.

Claramente si hemos armado todo esto es porque ambos datasets, entrenamiento y validación, son cruciales en la estructura de entrenamiento de nuestros modelos. El aporte del de entrenamiento es el ya visto en el **Capítulo 2**, con él se hace todo el proceso de feedforward, retropropagación y optimización. El dataset de validación, por otra parte, nos permite controlar el overfitting y guardar los modelos que progresan en este sentido, es decir, es la condición para guardar una nueva versión de la red, la reducción del error de validación.

No se ha hecho demasiado hincapié en ello pero todos estos datasets están tensorizados en formato `.pt`, como se puede ver en la carpeta `data`, y sus muestras están normalizadas. El mismo proceso se realiza por último para los datos completos, se cargan, se normalizan, se convierten en tensores archivos `.pt` distintos por canal. Si navegamos por la carpeta veremos 6 archivos, unos con sufijo 1 y otros con sufijo 2, cada uno correspondiente a una mitad del conjunto total ya que no podemos cargar todos los datos de golpe en nuestro sistema.

Pensando ya en la evaluación en otros tipos de datos, un proceso análogo puede realizarse (adaptando la parte de descarga de datos) para la obtención de los datasets `stars_dataset` y `NASA_dataset`, que nos servirán también para sacar conclusiones posteriormente.

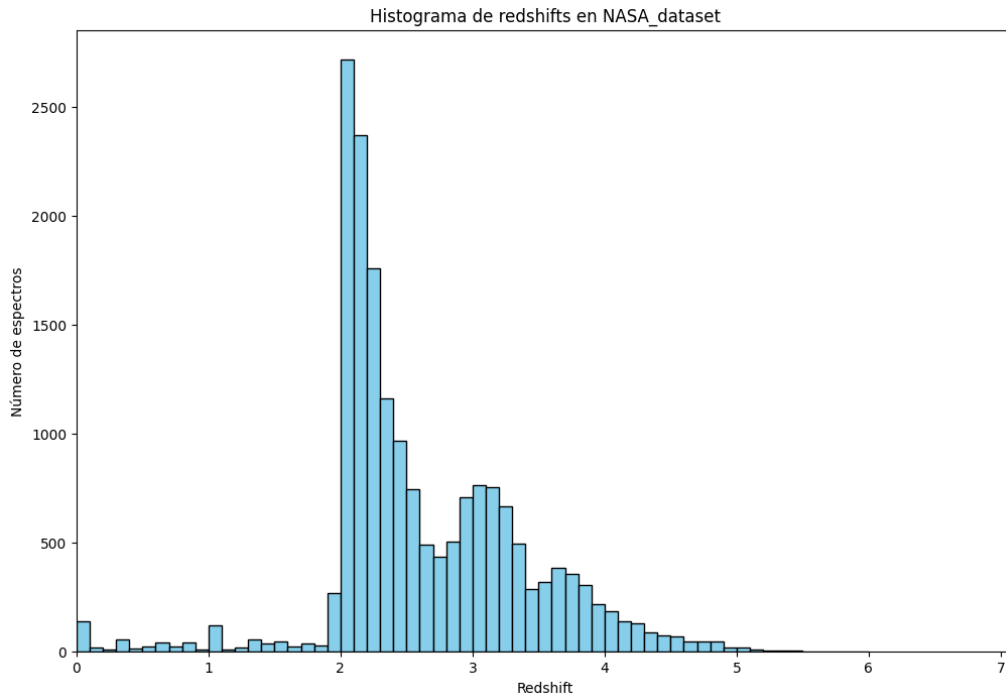


Figura 25: Distribución de las 18,255 muestras del dataset de la NASA en función de su redshift. La distribución para `stars_dataset` no es mostrada pues el redshift limita superiormente por 0.00462.

Si bien los archivos de la NASA también están en formato `.fits`, la extracción de los datos de longitud de onda y flujo varían ligeramente, pudiendo consultarse en el código.

Capítulo 4

Modelización del Problema

4.1 Estructura global

Todos los **notebooks** que podemos encontrar en el recurso online dedicados a modelos presentan la misma estructura a la hora de proceder, la cual se puede sintetizar a través de la siguiente imagen.

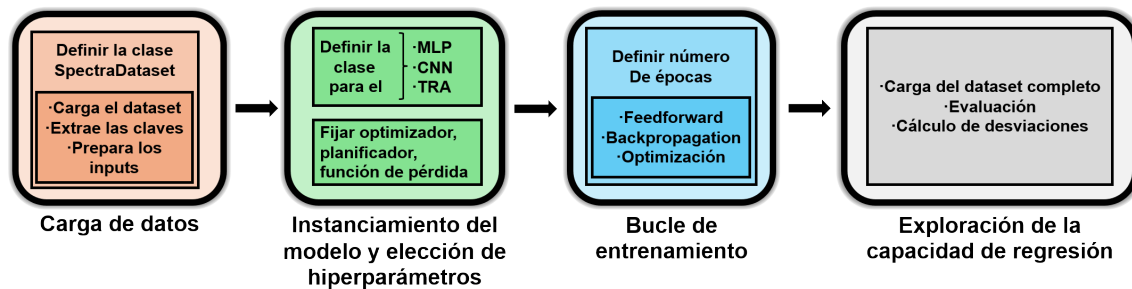


Figura 26: Resumen visual de la estructura del código.

A lo largo de este apartado explicaremos un poco de qué trata cada fase y las particularidades que presenta.

4.1.1 Carga de datos

El proceso de entrenamiento de cualquier modelo comienza con la carga de los datasets. Esto ya es un problema técnico en nuestro caso, debido al enorme espacio en memoria que necesitan. Para ello, crearemos una clase que encapsule los tensores de los datasets guardados previamente. A continuación, se define un método para devolver el número total de espectros, leyendo directamente la primera dimensión del tensor de flujo de manera que no se tengan que recorrer todos los datos.

Cuando se solicite un espectro concreto, solo se extraerán de la memoria los vectores de flujo y de longitud de onda correspondientes a ese índice. Gracias a este enfoque,

llamado comúnmente carga perezosa, en cada instante solo residen en la memoria los datos estrictamente necesarios para el lote en curso.

Para la fase de entrenamiento, los espectros se agrupan en lotes de tamaño fijo (32 muestras). Este valor se elige para equilibrar la varianza del estimador de gradiente (lotes demasiado pequeños pueden generar mucho ruido) con el consumo de memoria de la GPU.

4.1.2 Hiperparámetros generales

Al instanciar el modelo, se determina automáticamente si existe soporte para `CUDA`, el cual es un sistema del que dispone nuestra tarjeta gráfica que mejora enormemente la velocidad de los procesos de machine learning debido a que está muy orientado al trabajo en paralelo. Para hacer uso de él, debemos mover el objeto que queramos al dispositivo `CUDA`, que al fin y al cabo es la memoria gráfica, ya sea el propio modelo o los lotes.

Seguidamente se elige como función de pérdida el **error absoluto** (`L1Loss`), que da lugar al **error absoluto medio** (**MAE**) y mide la discrepancia promedio entre predicciones y valores reales sin elevar al cuadrado. Tras haberse experimentado anteriormente con el error cuadrático medio, que es más estándar, y haber comprobado que no hubiesen discrepancias, se elige realmente por comodidad de visualización y por no trabajar con errores minúsculos.

Para optimizar los parámetros se emplea `Adam`, que es el más estándar y da buenos resultados sin alterar demasiado cómo se actualizan los pesos mediante descenso de gradiente, con un learning rate inicial bajo de 10^{-4} y decaimiento de pesos. `Adam` combina momentos de primer y segundo orden para adaptar dinámicamente la tasa de aprendizaje de cada parámetro, acelerando la convergencia respecto al descenso de gradiente común [27].

Finalmente, se configura un planificador basado en `ReduceLROnPlateau`, lo cual se traduce a que se monitoriza la métrica de validación y, si no detecta mejora tras un número de épocas de “paciencia”, reduce la tasa de aprendizaje multiplicándola por un factor predefinido. Así se suaviza la trayectoria de optimización, pues cuando el progreso se estanca, los pasos de actualización disminuyen, favoreciendo un descenso más fino y evitando oscilaciones alrededor de mínimos locales.

4.1.3 Entrenamiento y validación

Todos los modelos están diseñados de forma que puedan seguir entrenándose desde donde se quedaron, guardando tanto la tasa de aprendizaje como el estado del optimizador y del planificador. El bloque de entrenamiento no es distinto al ya mencionado en el **Capítulo 2**, pues consiste en, para cada lote del dataset de entrenamiento, aplicar una primera fase feedforward en la que se obtienen las predicciones de la red a partir de los espectros y una segunda de retropropagación y actualización de los pesos mediante **Adam**.

Lo único a señalar es que, en el siguiente bloque, para cada lote del dataset de validación se lleva a cabo de manera análoga la fase feedforward, pasándole los resultados al planificador para que actúe si tiene que hacerlo y sirviendo como criterio para guardar o no el nuevo modelo actualizado.

Al terminar todas las épocas previstas, se representa la evolución de los errores del modelo y se realiza una evaluación final sobre el conjunto de entrenamiento y de prueba, para comprobar rápidamente que el modelo resultante es de buena calidad y posee una capacidad de generalización decente.

4.1.4 Poder de regresión

Para procesar los datos en dos mitades, se crea una clase análoga a la usada en la carga de los datasets, de la que ya hablamos al principio de este apartado, que lee por separado tres ficheros `.pt` (flujo, longitud de onda y redshift). Cada mitad usa rutas específicas que terminan en “_1” o “_2”. De esta forma se recorren ambos conjuntos por separado y, sin actualizar parámetros, se van acumulando predicciones y valores reales en listas que luego se concatenan y se comparan.

Una vez obtenidos los vectores de predicción y de referencia para cada mitad, se guardan, se realizan comparaciones y análisis interesantes como el MAE de todo el conjunto, el error en rangos altos y el número de redshifts negativos predichos. No es que z no pueda ser negativo en la práctica, pero los datasets utilizados en este proyecto son específicamente de muestras con redshift, es decir, se impuso la condición $z > 0$ durante la descarga.

Se presentan también en este bloque los mapas de calor que veremos en el siguiente capítulo. Estos mapas no son más que histogramas bidimensionales que nos indican de manera visual la distribución de los puntos `redshift verdadero` - `redshift predicho` en distintas zonas de interés. Se espera pues que estos se distribuyan lo más cerca posible de la recta identidad $y = x$, tomando como criterio matemático el **coeficiente de correlación lineal de Pearson** [29].

Esta parte es la que se considera realmente interesante, pues nos dice que tan bien se han llevado a la práctica las ideas presentadas en la creación de los conjuntos de entrenamiento, permitiéndonos conocer si los modelos son realmente útiles y podrían tener algún tipo de uso práctico.

4.2 Explicación de los modelos

4.2.1 Perceptrón multicapa

En este caso cada muestra consiste en un espectro formado por 5000 valores de flujo y 5000 valores de longitud de onda, concatenados en un único vector de 10000 elementos. Esta representación plana permite que las capas lineales de la MLP traten de forma conjunta toda la información espectral, sin asumir correlaciones locales en la secuencia sino confiando en la profundidad de la red para capturar relaciones complejas.

Se utiliza una arquitectura que encadena siete bloques, formados por capas lineales seguidas de la función de activación ReLU. El flujo de datos transita de la dimensión inicial de 10000 a 2048, luego a 1024, 512, 256, 128 y 64 neuronas respectivamente, generando progresivamente representaciones más compactas y abstractas de la señal original.

Al reducir el número de neuronas en cada etapa, la red fuerza al modelo a concentrar la información relevante en espacios de baja dimensionalidad, lo cual favorece la generalización y evita que memorice detalles superfluos.

Es justo antes de la capa de salida donde se inserta un mecanismo de dropout, de manera que en esta capa se apagan aleatoriamente la mitad de las neuronas en cada pasada época, lo que interrumpe rutas de propagación dominantes y obliga al modelo a distribuir el aprendizaje entre múltiples neuronas.

Esta última capa lineal colapsa las 64 características finales en un único valor escalar, nuestra predicción del redshift. De este modo, la MLP opera como un mapeo directo de un espacio de alta dimensión de entradas espectrales a una estimación puntual, sin recurrir a arquitecturas recurrentes ni de atención. La simplicidad de esta topología la hace especialmente eficiente de ejecutar y fácil de ajustar.

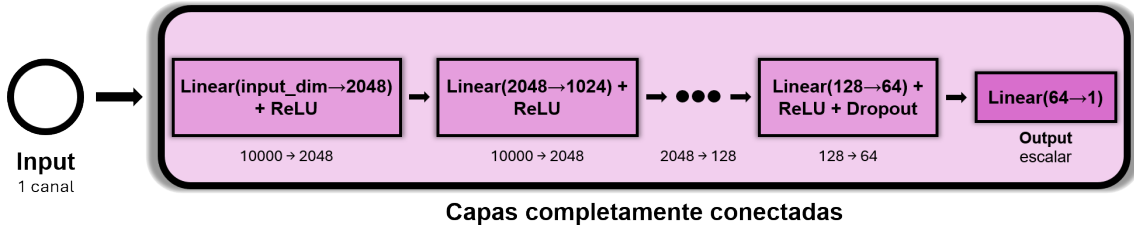


Figura 27: Estructura visual de los modelos MLP.

4.2.2 Red neuronal convolucional

Este modelo define una red neuronal convolucional que procesa espectros de dos canales (flujo y longitud de onda) de 5000 puntos cada uno, extrayendo características locales mediante operaciones de convolución.

En las primeras etapas, la red aplica cinco bloques idénticos de convolución unidimensional [13] con kernels de tamaño 3 y padding de 1, seguidos de la función de activación ReLU y de una operación de max-pooling con ventana 2. Esta manera de aplicar la convolución es idéntica a la antes mencionada, solo que más simple de entender ya que muchos conceptos encajan a la perfección con las imágenes mostradas (ahora sí se puede entender como una “línea de píxeles”, mientras que en la teoría se trabajaba con cuadrículas).

Cada convolución incrementa progresivamente el número de filtros (de 2 a 256), permitiendo aprender patrones de complejidad creciente, mientras que cada max-pooling reduce a la mitad la longitud de la señal, aportando invarianza local frente a pequeños desplazamientos en el espectro.

Tras cinco etapas de extracción y reducción, el tensor resultante de forma $(256, \frac{5000}{2^5})$ se aplana completamente a un vector de características de longitud 256×156 . A continuación, una capa lineal comprime esa representación en un espacio de 128 dimensiones con activación ReLU, obligando al modelo a concentrar la información más relevante antes de la predicción final en espacios de menor dimensionalidad.

El proyecto consta de los siguientes cinco modelos CNN independientes, accesibles mediante el [GitHub](#), basados en este principio.

- (I) En particular, el modelo descrito corresponde a los archivos `...0.ipynb`, que implementa la red convolucional básica.
- (II) La primera adición lógica consiste, tal como hicimos en los modelos MLP, en introducir dropout justo antes de la capa de salida, como se observa en los archivos `...1.ipynb`.
- (III) A continuación, evaluamos el margen disponible para ampliar el número de capas de convolución, implementado en los archivos `...2.ipynb`.
- (IV) En los archivos `...3.ipynb` incorporamos el mecanismo de **atención** utilizando **Multi-Head Attention** [11] basado en [3] entre las capas de convolución y las capas lineales, elemento clave que sustenta la idea del modelo final y constituye el núcleo de nuestro trabajo.
- (V) Finalmente, en los archivos `...4.ipynb` exploramos el efecto de aplicar un pequeño dropout a múltiples capas, configurando un valor reducido de dropout entre cada bloque de la arquitectura.

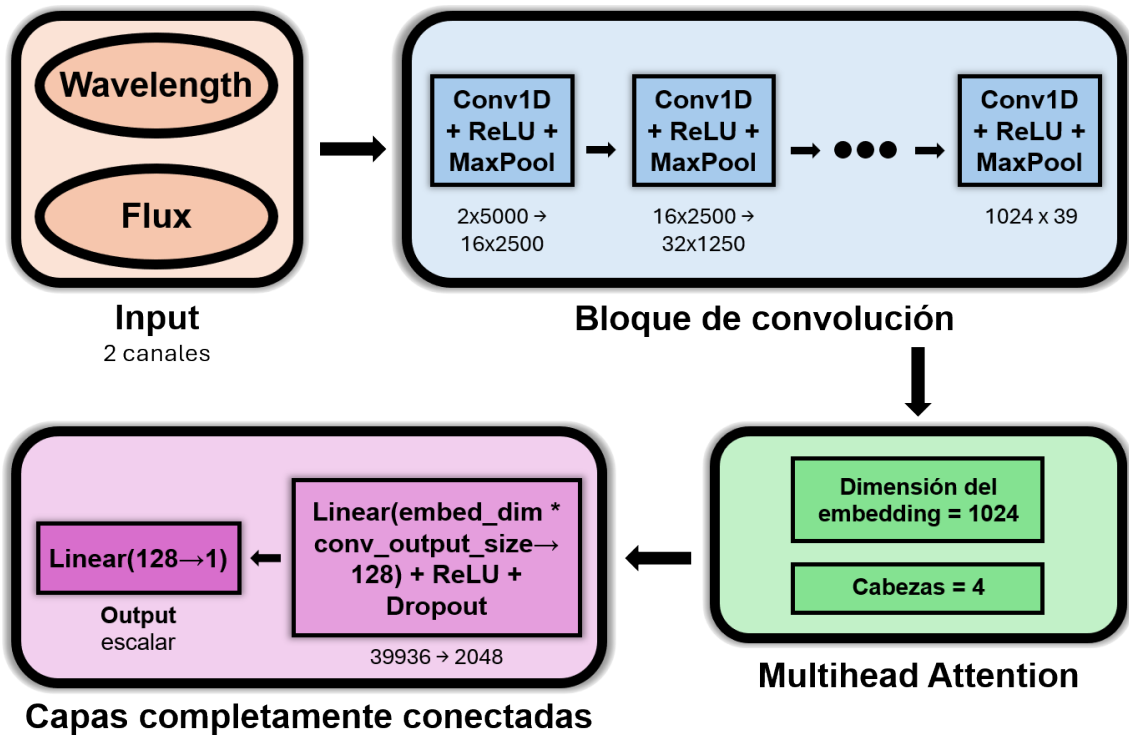


Figura 28: Estructura visual del modelo CNN. Este esquema representa la versión 3, pero recoge de manera clara todo lo necesario para comprender la estructura.

4.2.3 Transformer

Vamos a tratar de explicar de manera simple la estructura interna del modelo que hemos utilizado, apoyándonos en los conceptos definidos en la teoría. Partimos de una entrada de dimensiones 5000×5000 , que serán nuestros dos canales de flujo y longitud de onda nuevamente. Mediante convoluciones, convertimos estas dimensiones en $d \times L$, donde se elige $d = 128$, que será nuestra dimensión del espacio de características, y $L = 5000/16$.

Con esto hecho, se aplican ahora las codificaciones posicionales al resultado de este proceso, de dimensiones 128×312 , de forma que se define

$$\text{pe} \in \mathbb{R}^{1 \times L \times d}, \quad L = \frac{N}{16} = 312,$$

cuyas componentes vienen dadas por

$$\begin{cases} \text{pe}_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d}}\right), & pos = 0, \dots, L-1 \\ \text{pe}_{pos,2i+1} = \cos\left(\frac{pos}{10000^{2i/d}}\right), & i = 0, \dots, \frac{d}{2}-1 \end{cases}$$

se toma cada vector de características (de tamaño d) y se le añade, dimensión a dimensión, el vector de posición correspondiente.

A continuación, se define el codificador Transformer [14], formado por cuatro capas de atención con ocho cabezas. Esto puede imaginarse como una pequeña red neuronal densa que combina la información de todos los puntos de la secuencia de acuerdo al concepto de atención definido antes, captando relaciones más complejas que las convoluciones locales no pueden modelar solas.

Por último, se aplica pooling para resumir la secuencia en un solo vector representativo aplanado, de forma que las capas lineales finales puedan refinar esa representación para predecir un valor único.

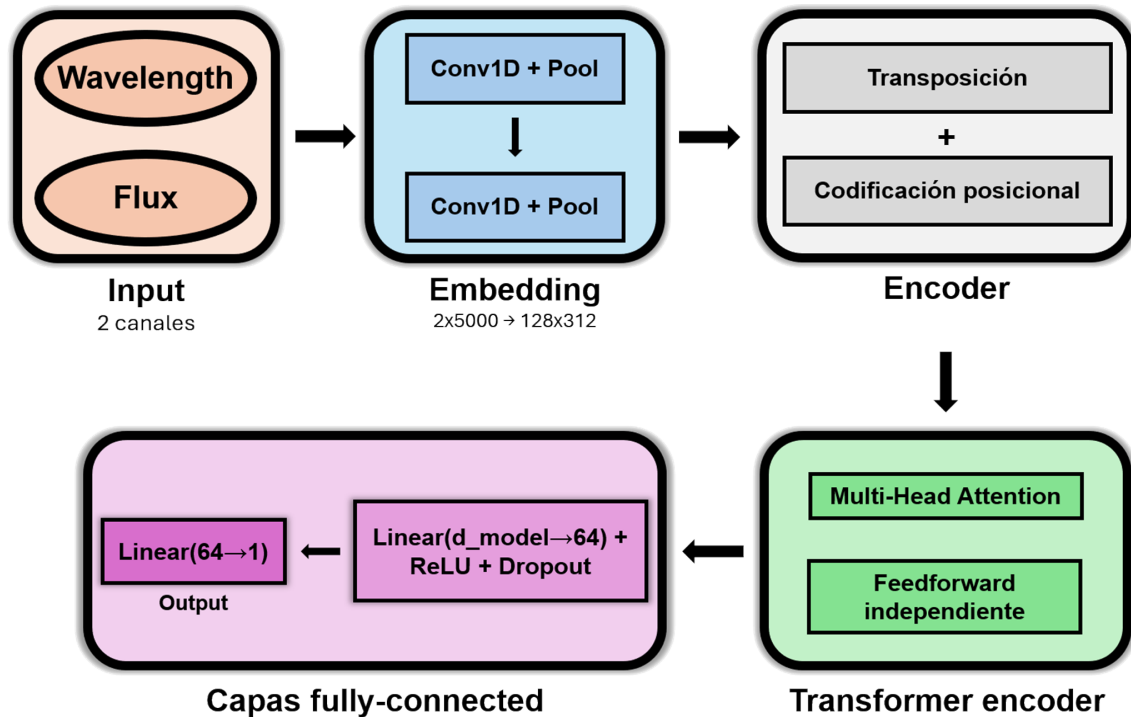


Figura 29: Resumen visual de nuestros modelos TRA, con los elementos introducidos anteriormente.

Capítulo 5

Resultados

5.1 Análisis comparativo

En este apartado se presentan los resultados de haber puesto a prueba todos los modelos en el conjunto completo de 3,372,890 espectros de la SDSS, que consta únicamente de las muestras de galaxias y cuásares que también se utilizaron para entrenar.

La última celda de cada archivo `.ipynb` permite al lector evaluar y visualizar un espectro aleatorio de la carpeta `spectrums`, además de permitirle seguir mucho mejor el procesamiento de un archivo `.fits` hasta llegar a los datos que permiten su uso en nuestros modelos.

Estos resultados tomarán la forma de gráficas y métricas de errores, que nos permitirán analizar las discrepancias entre los valores reales y los predichos por el modelo.

5.1.1 Los datasets de entrenamiento

Nos dedicamos en primer lugar a comparar los resultados obtenidos por los modelos entrenados con el dataset `_600k` y con el `_1M`, de forma que podamos descubrir si la justificación que fundamenta la construcción de este último dataset es consistente.

Es decir, las siguientes imágenes muestran las evaluaciones de todos y cada uno de los modelos entrenados con el dataset de `_600k` y, por separado, las de todos los modelos entrenados con el dataset `_1M`.

Hasta que se indique lo contrario, todos los modelos presentados se entrenaron hasta un número de épocas estándar. Por tanto, las imágenes mostradas a continuación corresponderán a modelos entrenados hasta la época número 30.

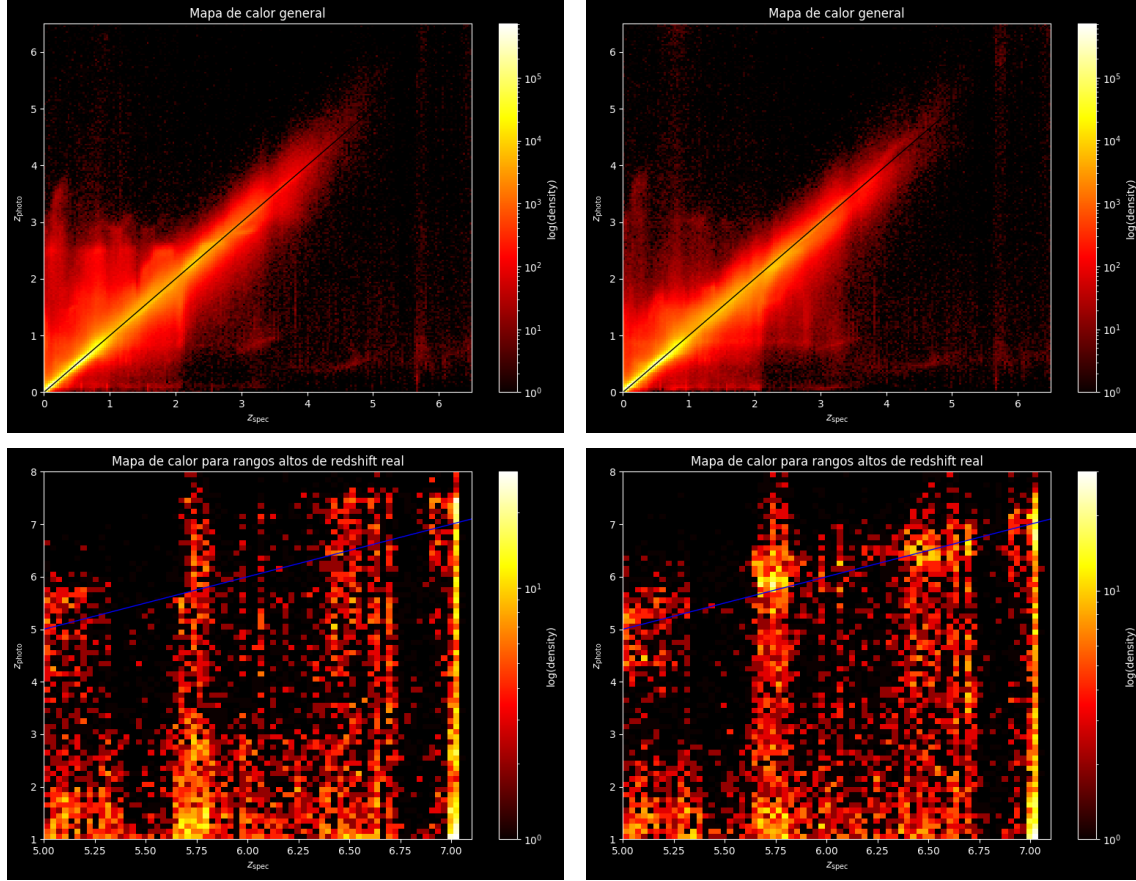


Figura 30: Mapas que muestran las diferencias entre usar los datasets `_600k` (izquierda) y los datasets `_1M` (derecha).

En la figura 30 mostramos en cada gráfico la densidad de puntos (z_{spec} , z_{photo}) en cada píxel en forma de mapa de calor, lo cual va a ser común durante toda esta sección. De acuerdo a los ejes, las imágenes superiores recogen prácticamente la totalidad de los puntos a representar, mientras que las inferiores comparan valores de redshift muy altos.

A nivel visual y en términos de errores y de coeficiente de correlación lineal, son bastante parecidas, aunque se puede apreciar una leve mejoría del modelo `_1M` en la zona de $z_{spec} > 5$, lo cual era el objetivo principal que se tenía, conseguir mejorar la precisión en redshifts altos (pocos datos) sin que esto introdujese sobreajuste en el modelo.

Esto queda más claro si, en lugar de observar las anteriores gráficas que salen de la representación de todas las predicciones de todos los modelos entrenados sobre el dataset completo, observamos el caso concreto del modelo TRA.

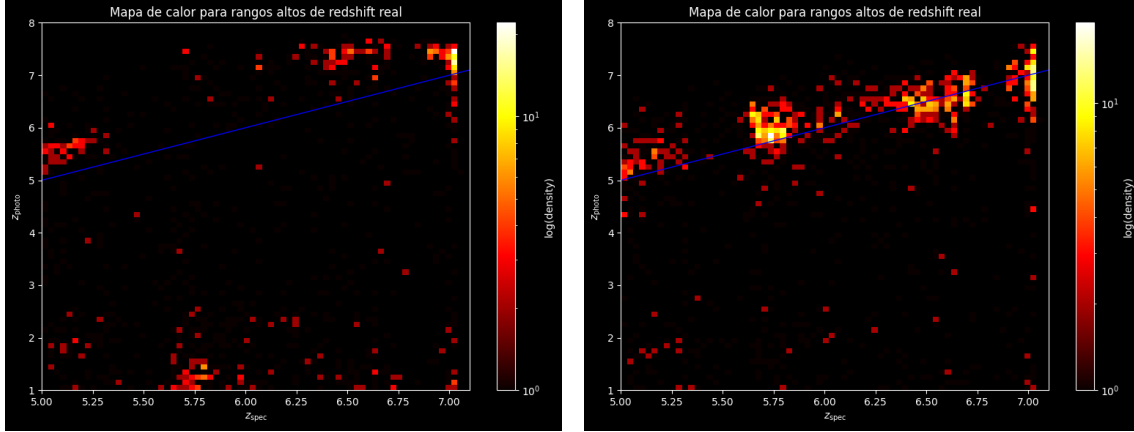


Figura 31: Mapas que muestran las diferencias concretas entre Model TRA 600k (izquierda) y Model TRA 1M (derecha) en redshifts altos.

En la figura 31 se observa muy claramente cómo lo que antes era una nube de puntos aleatoria en redshifts altos, ahora tiene un carácter más lineal. Esto se puede comprobar calculando el error absoluto medio, el cual resulta ser de 3.7 para el entrenamiento con el dataset de _600k frente a 1.8 obtenido con el de _1M.

Por todo esto, a la hora de presentar los resultados obtenidos mostraremos únicamente los de los modelos _1M, ya que solo nos introducen mejoras en nuestros modelos.

5.1.2 El modelo MLP

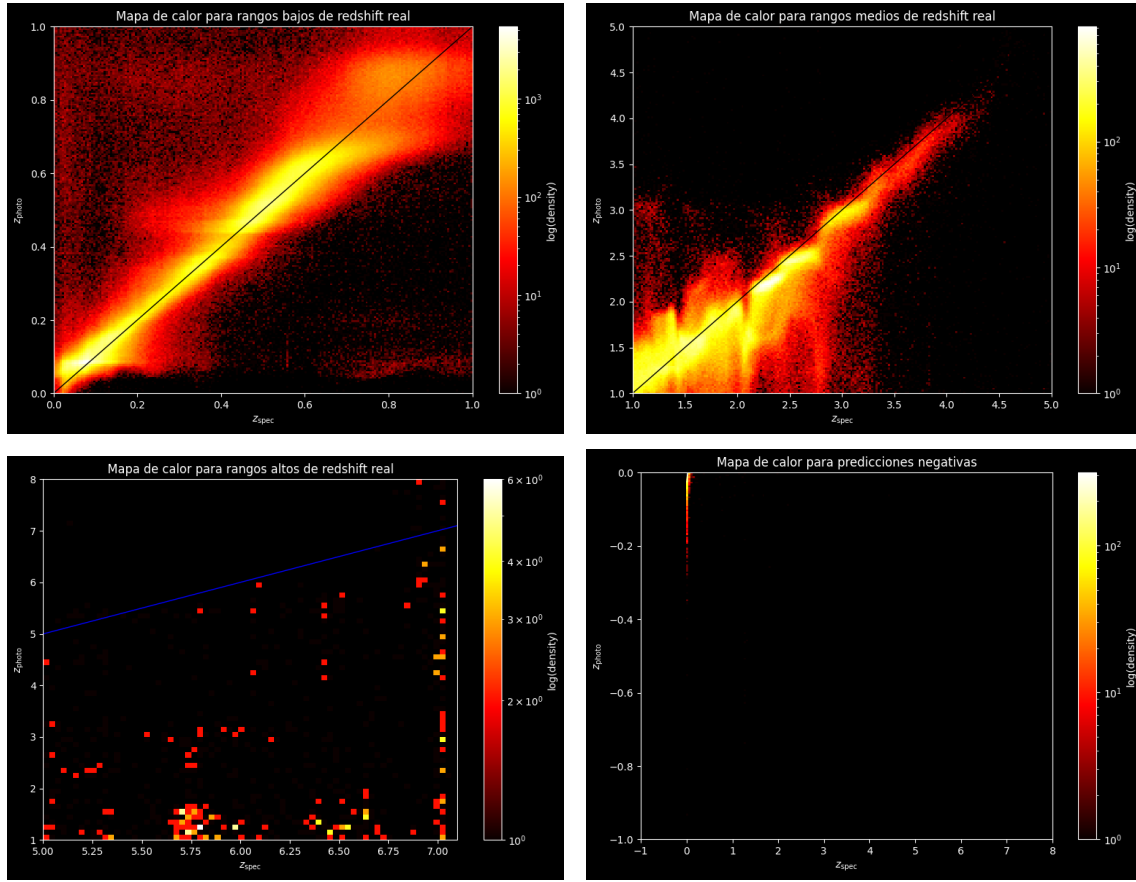


Figura 32: Mapas que muestran los resultados al evaluar el modelo MLP en el dataset completo, por rangos.

En primer lugar, el modelo MLP nos dio un coeficiente de correlación lineal de 0.477 y un error absoluto medio de 0.128, con un total de 2,181 predicciones de redshifts negativos. Estos resultados tan pobres, reforzados por el mapa de calor, nos llevaron a concluir que el modelo no estaba consiguiendo abstraer de manera correcta los patrones inherentes al redshift.

Igualmente, no hay que despreciar la utilidad de estos resultados, pues sirvieron inicialmente para comprobar que la manera de definir los datasets y de entrenar el modelo era acertada, pues no son predicciones radicalmente malas.

5.1.3 Los modelos CNN

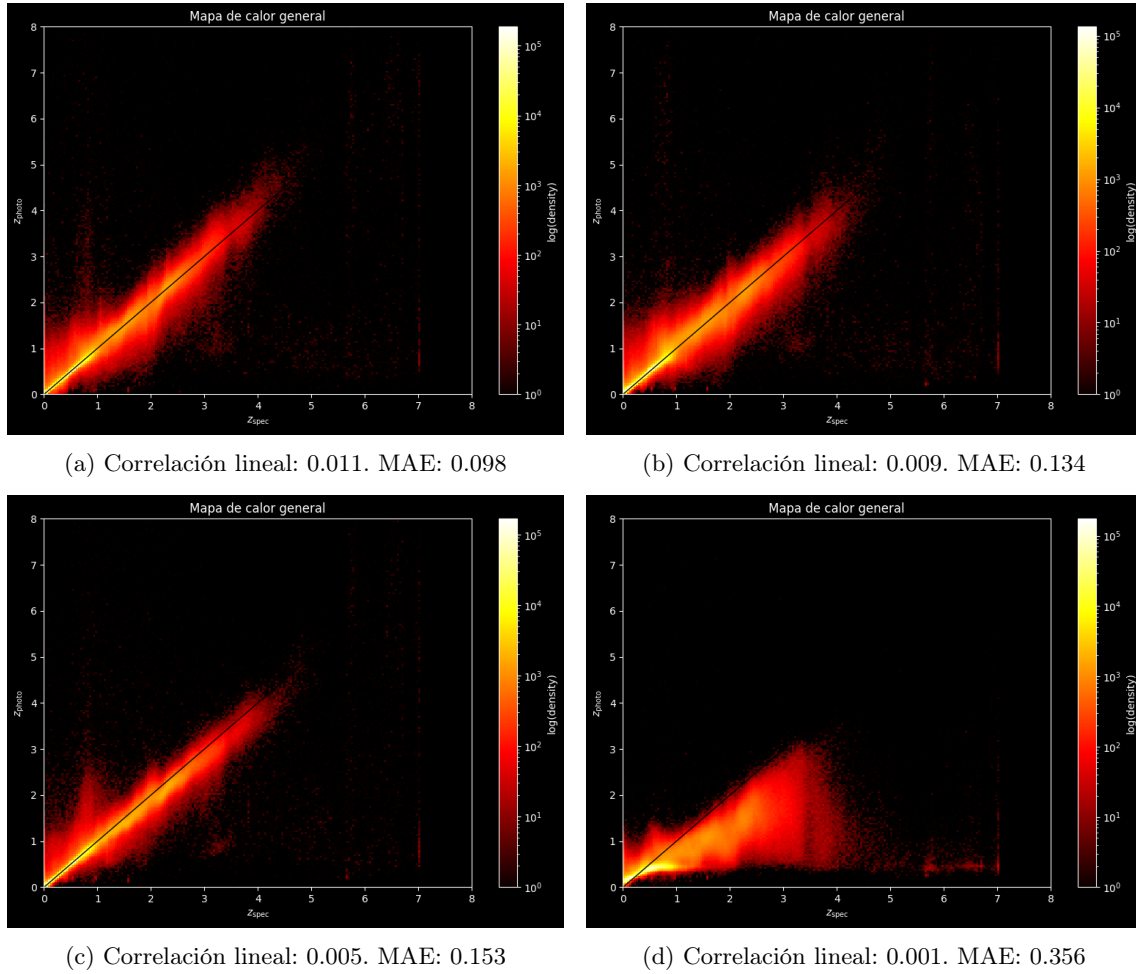


Figura 33: Mapas que muestran los resultados de los distintos modelos $(a) \equiv 0$, $(b) \equiv 1$, $(c) \equiv 2$ y $(d) \equiv 4$.

Recordamos brevemente que la variante 0 era la red convolucional simple, en la variante 1 introducimos dropout antes de la capa de salida, en la variante 2 añadimos capas convolucionales y en la variante 4 añadimos un pequeño dropout entre capas convolucionales.

Si bien a nivel de tratamiento de errores no mejoran demasiado al modelo MLP presentado, se aprecia visualmente algo menos de dispersión. La introducción de más capas y de dropout perjudica levemente los resultados a la hora de analizar la capacidad de generalización, que en general son bastante malos y no se aprecian diferencias excesivas entre ellos.

Sin embargo, una conclusión clara que podemos sacar es que introducir dropout en todas las capas hace que el modelo funcione realmente mal, como podemos apreciar en (d) de la figura 33.

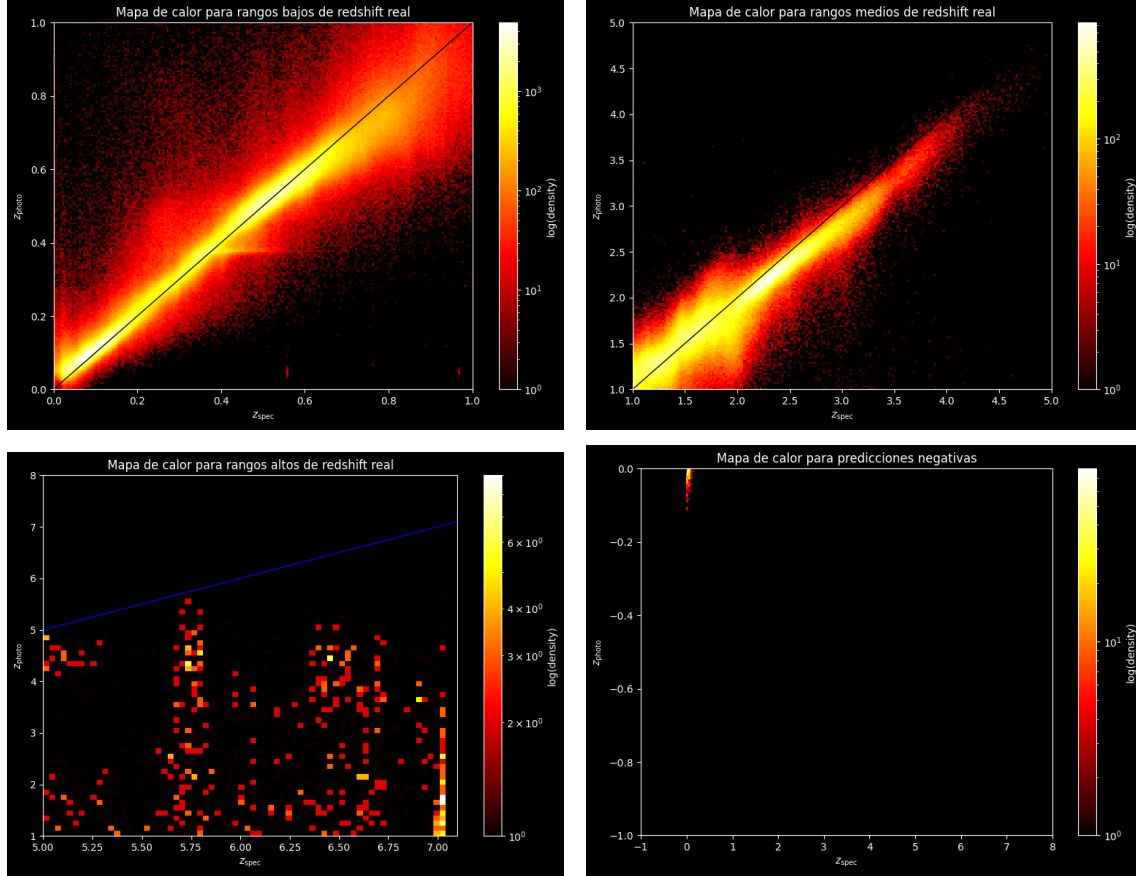


Figura 34: Mapas que muestran los resultados al evaluar la tercera variante del modelo CNN en el dataset completo, por rangos.

De todas formas, la reducción dimensional o proceso de creación del embedding se hace necesaria para el modelo 3, cuyos resultados se muestran en la figura 34, pues es una fase previa que se ha de realizar antes de utilizar el módulo de atención `MultiheadAttention` [11], el cual mejora enormemente el rendimiento alcanzando una correlación lineal de 0.967 y un error medio de 0.073, prediciendo un total de 565 redshifts negativos.

Lo que más llama la atención es la incapacidad del modelo para predecir los redshifts de rangos altos, $z \geq 5$, pues en este caso se está tratando con errores absolutos de 4.007.

5.1.4 El modelo Transformer

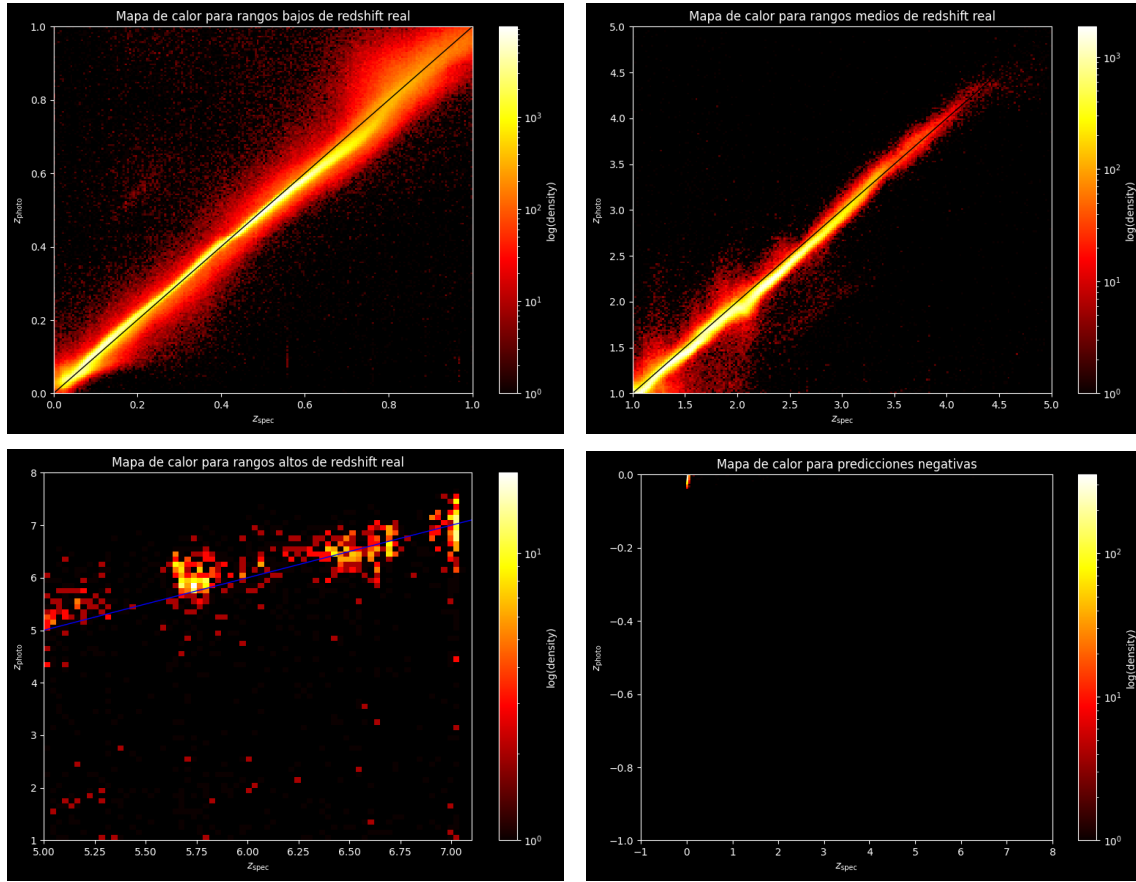


Figura 35: Mapas que muestran los resultados al evaluar el modelo TRA en el dataset completo, por rangos.

Los resultados de implementar el modelo presentado por [3] son muy satisfactorios, como podemos observar si comparamos las figuras 34 y 35. El modelo es capaz de generalizar aún más a partir de los datos suministrados, cumpliendo con la labor de relacionar picos distantes y asociarlos a las líneas de emisión de ciertos elementos como se presentó en la teoría del redshift del **Capítulo 2**.

Concretando, estamos hablando de un coeficiente de correlación lineal de 0.965, un error medio de 0.039, prediciendo un total de 1,570 redshifts negativos, los cuales podemos observar en la imagen (d) de la figura 35 que son mucho más cercanos a 0 que en los anteriores casos.

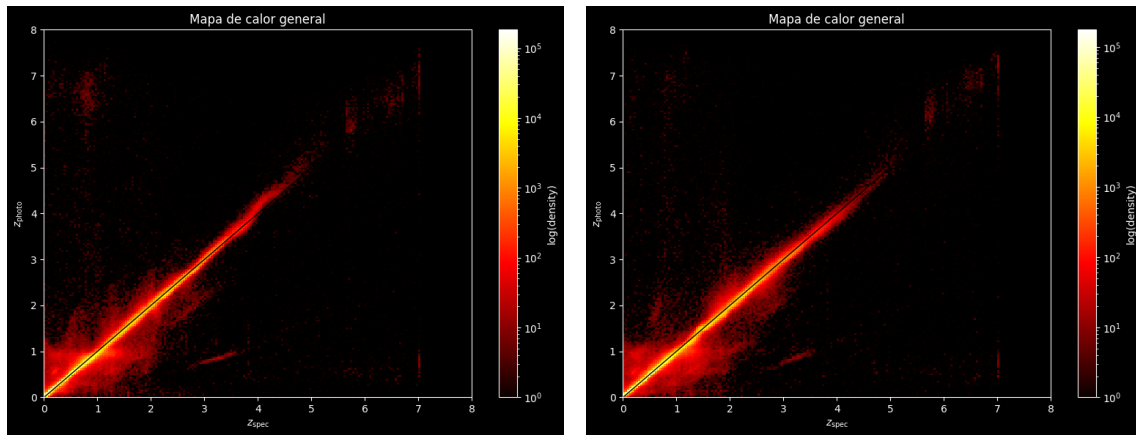
5.1.5 Influencia de la normalización

Para este apartado, a todos y cada uno de los archivos `.pt` se les aplicó un proceso en el cual se revirtió la normalización inicial expuesta anteriormente, basada en `StandardScaler` [30]. Seguidamente, se utilizó una normalización más natural, la cual consistió en

- (I) centrar el espectro desplazando las longitudes de onda para que el valor medio quede en cero.
- (II) dividir todas las longitudes de onda por su valor máximo, en valor absoluto, escalándolas al rango $[-1, 1]$.
- (III) aplicar el mismo procedimiento al flujo, normalizándolo de modo que su valor máximo sea 1.

Podemos comparar fácilmente cómo esta normalización es menos agresiva y abstracta en las figuras 22 y 23 antes mostradas, pues ahora el espectro normalizado mantendría exactamente la misma forma del espectro original. Esto nos brinda una manera de trabajar más directa y sencilla.

En la siguiente figura se comparan las evaluaciones de ambos modelos en el dataset completo.



(a) Resultados presentados anteriormente del modelo TRA sin renormalizar, con 30 épocas de entrenamiento.

(b) Resultados presentados anteriormente del modelo TRA con la nueva normalización, con 30 épocas de entrenamiento.

Figura 36: Efecto de la renormalización.

El modelo con la renormalización mejora levemente con respecto al anterior, con una correlación lineal de 0.975 y un error medio de 0.030, prediciendo un total de 665 redshifts negativos. Igualmente, no observamos variaciones extremadamente significativas, lo cual es lo que buscábamos ya que este cambio de normalización está motivado por el problema del **data drift** o cambios radicales en la forma y calidad de los inputs, que se expone un poco más abajo.

5.2 Problemáticas

5.2.1 Épocas

Tras haber concluido que el modelo Transformer era el mejor de todos los que se habían entrenado en igualdad de condiciones, el siguiente paso natural era seguir entrenándolo para ver si se conseguían resultados mucho mejores con el paso de las épocas. Por tanto, partiendo de estas 30 épocas, se continuó entrenando el modelo con el mismo dataset `_1M` para observar qué sucedía.

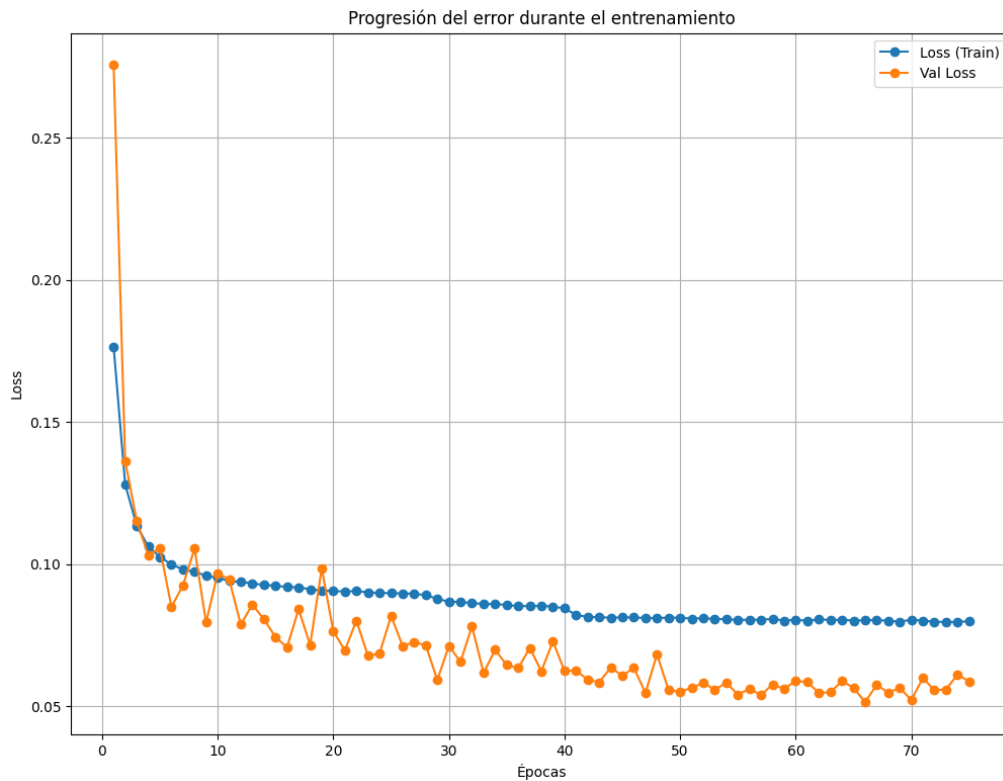


Figura 37: Evolución del error del modelo TRA con las épocas.

Los resultados, pese a que ya se esperaba que no mejorarían demasiado debido a la alta precisión que habíamos conseguido, fueron bastante pobres. Conseguimos mejorar en tan solo dos milésimas el coeficiente de correlación, de 0.967 en este caso, y reducir un poco el error medio hasta 0.031, prediciendo un total de 1,570 redshifts negativos.

Si bien es verdad que el modelo progresa, no podemos concluir que lo hará mucho más, de forma que podemos estar ante un posible estancamiento y haber llegado a una de las mejores versiones posibles del modelo. Esto podría ser un problema como veremos en el siguiente apartado.

5.2.2 Rangos bajos

Para medir y exponer la capacidad del modelo TRA en rangos muy bajos de redshift, $z \approx 0$, se hizo uso del dataset de estrellas presentado en el Capítulo 3.

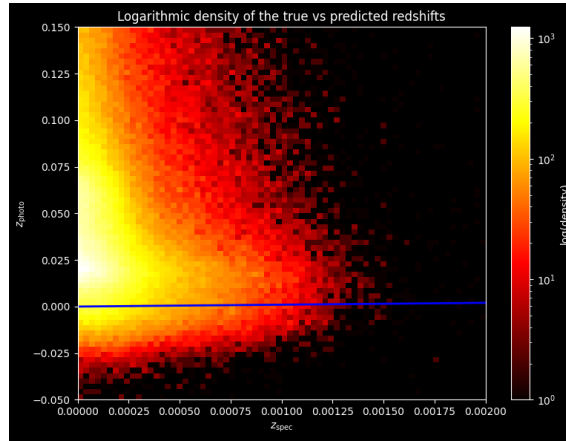


Figura 38: Evaluación del modelo TRA, sin renormalizar en el dataset de estrellas.

Como podemos ver, los resultados son paupérrimos. La línea azul, como siempre, es la función identidad que utilizamos siempre en rangos bajos de redshift y alrededor de la cual se debería agrupar la densidad de puntos. Lo que observamos, sin embargo, es una zona relativamente localizada alrededor de esta pero una dispersión muy grande hacia predicciones de redshift mayores.

En este caso estamos ante una correlación lineal de 0.0884 y error absoluto de 0.627859 cuando, como presentamos anteriormente, el valor máximo de redshift en estrellas es de 0.00462.

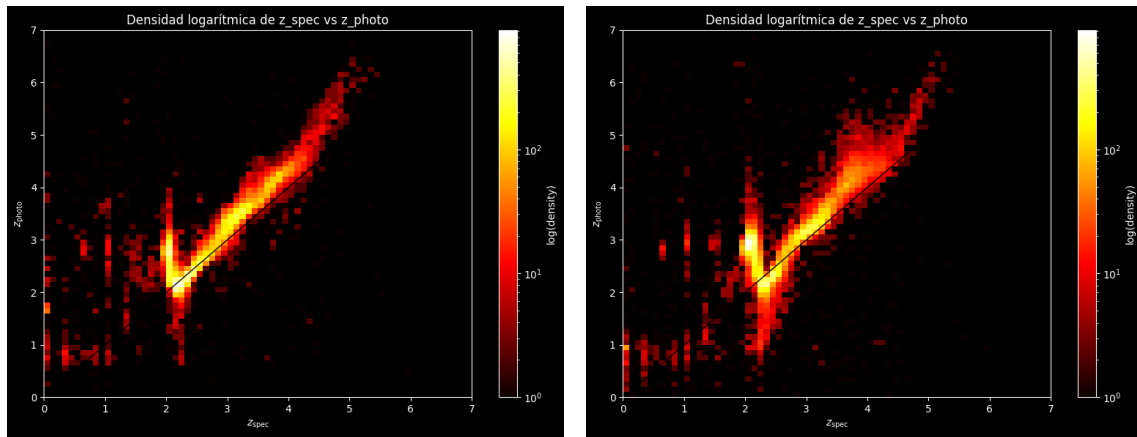
Si bien se ha elegido un dataset formado por objetos distintos, esto es algo que ya sucedía en menor medida anteriormente con galaxias y cuásares. Debido al tamaño de los píxeles elegido para representar las distribuciones anteriores, es posible que haya pasado desapercibido este fenómeno, ligado al concepto de error relativo.

Todos los MAE presentados anteriormente se mantienen uniformemente a lo largo de todo el rango de z_{spec} , por lo que, claramente, al tener que predecir valores cercanos a 0 el error relativo aumenta.

Si bien esto puede parecer obvio, tratar con objetos de bajo redshift para los que se pueden requerir precisión de hasta sexto decimal es un problema que no resuelve este modelo general y tiene cabida en la argumentación, sugiriéndose la exploración de modelos entrenados específicamente para rangos bajos y, sobre todo, para estrellas, de forma que podamos subsanarlo.

5.2.3 Data drift

Que los datos que le pasamos al modelo en forma de espectros sean uniformes depende fuertemente del tipo de instrumental utilizado y de la calidad de los mismos. Aquí utilizamos datos del NED para observar este fenómeno, el cual puede llegar a ser un problema ya que, al cambiar la normalización por una más general, tampoco lo conseguimos subsanar.



(a) Fenómeno del data drift en el modelo TRA antes de la renormalización.

(b) Fenómeno del data drift en el modelo TRA con la renormalización aplicada.

Figura 39: Detalle del data drift.

El modelo sin renormalización presenta un coeficiente de correlación lineal de 0.812 y un MAE de 0.335, mientras que el modelo entrenado con datos renormalizados rinde aún peor con un coeficiente de Pearson de 0.72 y un MAE de 0.483.

A continuación se presenta una imagen de un espectro renormalizado de esta base de datos para el que obtuvimos un error relativo de 3.34, sobre el cual vamos a comentar una serie de cosas.

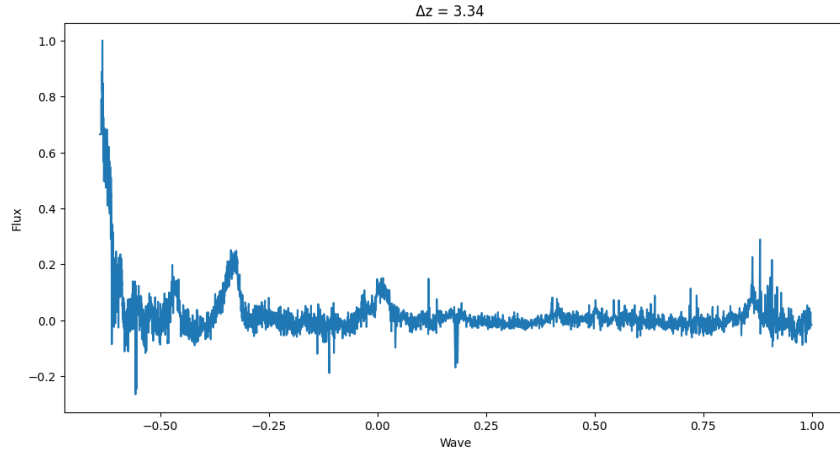


Figura 40: Espectro de la base de datos del NED con redshift $z = 2.07$.

Lo primero que nos salta a la vista es que, si nos fijamos a la izquierda del eje de la longitud de onda, el espectro no ha llegado ni tan siquiera a valores de longitud de onda de -0.75 tras la normalización. Esto ocurre debido a que, si bien el proceso de interpolación, utilizado para unificar las entradas y que tengan todas tamaño 5000×5000 es el mismo, como este tipo de datos apenas superan de base las dimensiones 1200×1200 hay que aplicar este proceso bastantes veces, generando demasiado sesgo en los picos y desplazando el espectro.

Esto hace que, como hay un pico demasiado grande a la izquierda, se introduzcan una gran cantidad de valores tanto de longitud de onda como de flujo en esa zona, de forma que la media se aleja mucho del valor que se esperaría en condiciones ideales de la semidiferencia entre el máximo y el mínimo valor. Este fenómeno no ocurría con los datos de la SDSS debido a que se añadían de manera bastante uniforme los puntos al interpolar, pues se partía aproximadamente de 4400×4400 .

Aclarar que la interpolación funciona calculando una única vez los mayores cambios en flujo cada vez que se aplica, de forma que, si hay un pico muy abrupto, aplicarla 10 veces significaría meter 10 puntos en esa zona, mientras que aplicarla una vez introduciría un único punto.

Si bien se podría haber pensado otra normalización y cambiar completamente la manera de tratar la unificación de las entradas, la mejor manera de tratar este problema será siempre desde la toma de datos, pues mientras no se tenga absolutamente controlado esto hay muchas variables que pueden influir, como la capacidad de recepción de luz del sensor, la precisión de la red de difracción o incluso el rango del espectro electromagnético que se es capaz de medir.

5.3 Discusión

En esta sección reflexionamos sobre el desempeño general de los modelos y el impacto de las decisiones de diseño, condensando los resultados obtenidos en un análisis más claro.

Al comparar los modelos entrenados con distintas cantidades de muestras, se aprecia que incrementar la base de datos aporta beneficios especialmente en las regiones del espectro donde antes había escasez de información. Sin embargo, este aumento no introduce un sesgo excesivo ni deriva en un ajuste demasiado estricto. Más bien, refuerza la estabilidad de las predicciones bajo condiciones de baja densidad de datos. Esta observación justifica centrarse en el conjunto ampliado para todas las evaluaciones posteriores, ya que maximiza la robustez sin comprometer la generalización.

El modelo de perceptrón multicapa aporta una línea base útil para validar el flujo de trabajo de procesamiento y entrenamiento. Aunque su capacidad para capturar la complejidad espectral resulta limitada, estos primeros resultados confirmaron que el preprocesado y la definición de los conjuntos de datos eran adecuados. Su principal aportación radica en demostrar que, incluso con una arquitectura sencilla, el método es funcional y sirve de punto de partida para arquitecturas más sofisticadas.

Las diferentes configuraciones de CNN exploradas muestran cómo introducir dropout o capas adicionales puede reducir ligeramente la dispersión en nuestro caso, pero una regularización excesiva termina por degradar el ajuste. Esto revela también un delicado equilibrio entre la profundidad del modelo y la capacidad de aprender patrones relevantes, pues demasiados filtros pueden hacer que el modelo memorice demasiado. Estas conclusiones sugieren explorar mecanismos intermedios, como atenciones locales, para potenciar la extracción de características sin caer en sobrecarga de parámetros.

La fase de embedding, previa al uso de atención, demostró ser esencial para compactar la información sin perder detalles críticos. Al incorporar un módulo de atención global, el modelo logra asociar eficazmente picos espectrales distantes con los patrones de emisión de ciertos elementos de manera más eficaz. Este hallazgo subraya la importancia de separar la extracción de características locales de la agregación global, sentando las bases para arquitecturas híbridas que combinen lo mejor de las CNN y los Transformers.

La adopción de la arquitectura basada completamente en el artículo permitió capturar estas relaciones aún mejor, mejorando notablemente la coherencia de las predicciones generales. Esta capacidad para conectar eventos de larga distancia espectral refuerza la hipótesis teórica de que las atenciones globales son adecuadas para la tarea de redshift, abriendo la puerta a optimizaciones que reduzcan la carga computacional manteniendo la expresividad del núcleo de atención.

El cambio de un escalado estadístico abstracto a una renormalización más directa, centrada en conservar la forma original del espectro, se tradujo en una mejora perceptible del ajuste sin alterar drásticamente la tendencia global. Este enfoque menos agresivo se propone para manejar de manera más controlada los efectos de variaciones instrumentales, ofreciendo mayor resistencia a cambios en la calidad de los datos de entrada.

Prolongar el número de épocas mostró una fase de meseta en el progreso, lo que indica que se está alcanzando una versión próxima al óptimo bajo las condiciones actuales. Esto sugiere explorar alternativas como ajustes dinámicos distintos de la tasa de aprendizaje o entrenamiento cíclico antes que simplemente aumentar las iteraciones.

Al evaluar objetos con valores muy cercanos a cero, emerge un reto de precisión relativa: un mismo nivel de error absoluto genera discrepancias significativas en las zonas de menor redshift. Para abordar esto, se propone entrenar modelos específicos para estos rangos reducidos, o bien emplear funciones de pérdida adaptativas que penalicen con mayor fuerza las desviaciones cerca de cero, pudiendo quizás lograr mejores errores relativos en estos casos concretos.

La aplicación del modelo a datos procedentes de metodologías diferentes puso de manifiesto un sesgo inducido por la interpolación y la resolución variable de los espectros. Este fenómeno resalta la necesidad de integrar estrategias de adaptación de dominio, ya sea mediante técnicas de calibración previa o entrenamiento con muestras que reflejen la diversidad de procesos instrumentales.

En conjunto, estos análisis apuntan a que la combinación de un dataset amplio, arquitecturas basadas en atención y normalizaciones conservadoras ofrece la mejor ruta hacia predicciones de redshift robustas. No obstante, los desafíos en zonas extremas del espectro y la variabilidad de los datos instrumentales subrayan la importancia de continuar explorando estrategias de especialización y adaptación.

Capítulo 6

Conclusiones

Cerramos el trabajo mediante una serie de comentarios en términos generales de los objetivos que este ha cumplido. En el plano teórico, este trabajo ha profundizado en la interpretación física de las líneas de emisión y en la teoría del redshift que se ha ido construyendo alrededor del redshift, conceptos que aparecen a lo largo de la titulación. Por otra parte, la investigación y desarrollo de la teoría del aprendizaje automático aquí presentada, campo en el que no se disponía de conocimiento previo, ha permitido desarrollar unas bases teóricas sólidas sobre sus fundamentos.

La falta de precisión en los modelos más simples y famosos, como los MLP y los CNN, obligó a la exploración de alternativas que desembocaron en uno de los artículos más interesantes y relevantes en el mundo de la inteligencia artificial, el cual, pese a ser la arquitectura estrella de los modelos de lenguaje, ha podido ser adaptado a un problema tan distante como lo es el desarrollado durante este documento, demostrando la capacidad de los modelos basados en atención.

En el plano práctico, se ha demostrado que este último modelo de atención puede servir no solo para predecir, sino también para validar las mediciones de catálogos como los de la Sloan Digital Sky Survey debido a la alta precisión que presenta. Al contrastar las predicciones automáticas con los valores publicados, es posible detectar anomalías que alerten sobre posibles errores instrumentales o de procesado, ofreciendo así una herramienta adicional de control de calidad en cartografía cosmológica.

Finalmente, comentar que los modelos aquí presentados son, hasta cierto punto, simples desde el punto de vista de alguien especializado en machine learning, siendo desconocido si hay técnicas o acercamientos mejores al problema presentado. Lo que queda claro con este trabajo es que, de nuevo, la inteligencia artificial se abre camino por senderos en los que no se esperaba que fuese a tener cabida.

Bibliografía

- [1] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. ISBN: 9780262035613. URL: <https://www.deeplearningbook.org/>.
- [2] J. David Nightingale James Foster. *A Short Course in General Relativity*. 3.^a ed. New York: Springer Science, ene. de 2005.
- [3] Ashish Vaswani et al. «Attention Is All You Need». En: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. arXiv: 1706.03762 [cs.CL].
- [4] C. O’Raifeartaigh. «The contribution of VM Slipher to the discovery of the expanding universe». En: *Proceedings of the Origins of the Expanding Universe Conference*. Flagstaff, Arizona, sep. de 2012. URL: <https://philarchive.org/archive/RAITC0-3> (visitado 07-05-2025).
- [5] Matthew D. Schwartz. *Lecture 21: The Doppler Effect*. URL: <https://scholar.harvard.edu/files/schwartz/files/lecture21-doppler.pdf> (visitado 26-05-2025).
- [6] Sloan Digital Sky Survey Collaboration. *SkyServer DR1: Get Image by Plate*. 2003. URL: https://skyserver.sdss.org/dr1/en/tools/getimg/plate.asp?utm_source=chatgpt.com (visitado 07-05-2025).
- [7] Anass El Fares. *A better understanding how a CNN works (Code available)*. Medium. 20 de oct. de 2021. URL: https://medium.com/@El_Fares_Anass/a-better-understanding-how-a-cnn-works-code-available-e880f9a338dc (visitado 28-05-2025).
- [8] Swinburne University of Technology. *Gravitational Redshift*. COSMOS, Swinburne University of Technology. URL: <https://astronomy.swin.edu.au/cosmos/g/Gravitational+Redshift> (visitado 28-05-2025).
- [9] NASA Science. *Hubble Gravitational Lenses*. NASA Science. URL: <https://science.nasa.gov/mission/hubble/science/science-behind-the-discoveries/hubble-gravitational-lenses/> (visitado 28-05-2025).
- [10] Robert Hogan, Malcolm Fairbairn y Navin Seeburn. «GAz: A Genetic Algorithm for Photometric Redshift Estimation». En: (2014). arXiv preprint. arXiv: 1412.5997 [astro-ph.IM].
- [11] PyTorch Contributors. *torch.nn.MultiheadAttention [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html> (visitado 28-05-2025).
- [12] PyTorch Contributors. *torch.nn.Dropout [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html> (visitado 28-05-2025).
- [13] PyTorch Contributors. *torch.nn.Conv1d [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv1d.html> (visitado 28-05-2025).
- [14] PyTorch Contributors. *torch.nn.TransformerEncoderLayer [Online Documentation]*. 2025. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.TransformerEncoderLayer.html> (visitado 28-05-2025).

- [15] PyTorch Team. *torch.nn.ReLU*. PyTorch. 2024. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html> (visitado 28-05-2025).
- [16] Sloan Digital Sky Survey Collaboration. *The Sloan Digital Sky Survey*. Apache Point Observatory y the Astrophysical Research Consortium. 2025. URL: <https://classic.sdss.org/home.php> (visitado 28-05-2025).
- [17] Sloan Digital Sky Survey Collaboration. *Understanding the Imaging Data*. 2025. URL: https://www.sdss.org/dr16/imaging/imaging_basics/ (visitado 28-05-2025).
- [18] Sloan Digital Sky Survey Collaboration. *Filters*. 2025. URL: <https://www.sdss4.org/instruments/camera/#Filters> (visitado 28-05-2025).
- [19] Sloan Digital Sky Survey Collaboration. *The Spectroscopic Pipeline*. 2012. URL: <https://www.sdss3.org/dr9/spectro/pipeline.php> (visitado 28-05-2025).
- [20] Sloan Digital Sky Survey Collaboration. *CasJobs – Batch SQL Queries and MyDB*. 2025. URL: <https://casjobs.sdss.org/> (visitado 28-05-2025).
- [21] Sloan Digital Sky Survey Collaboration. *Science Archive Server (SAS) Data Access*. 2025. URL: https://www.sdss.org/dr17/data_access/tools/ (visitado 28-05-2025).
- [22] NASA/GSFC FITS Support Office. *FITS Standard Documentation*. 2025. URL: <https://fits.gsfc.nasa.gov/> (visitado 28-05-2025).
- [23] Sloan Digital Sky Survey Collaboration. *Redshifts, Classifications and Velocity Dispersions*. 2025. URL: <https://www.sdss.org/dr17/algorithms/redshifts/> (visitado 28-05-2025).
- [24] Sloan Digital Sky Survey Collaboration. *BOSS Pipeline Documentation*. 2022. URL: <https://www.sdss.org/dr18/software/pipelines/boss/> (visitado 28-05-2025).
- [25] NASA/IPAC Extragalactic Database. *NASA/IPAC Extragalactic Database: Home*. 2025. URL: <https://ned.ipac.caltech.edu/> (visitado 28-05-2025).
- [26] Astroquery Developers. *NED Queries (astroquery.ipac.ned)*. n.d. URL: <https://astroquery.readthedocs.io/en/latest/ipac/ned/ned.html> (visitado 28-05-2025).
- [27] PyTorch Contributors. *Adam — PyTorch 2.7 documentation*. n.d. URL: <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html> (visitado 28-05-2025).
- [28] PyTorch Contributors. *torch.optim — PyTorch 2.7 documentation*. n.d. URL: <https://pytorch.org/docs/stable/optim.html> (visitado 28-05-2025).
- [29] NumPy Developers. *numpy.corrcoef — NumPy v2.2 Manual*. n.d. URL: <https://numpy.org/doc/2.2/reference/generated/numpy.corrcoef.html> (visitado 28-05-2025).
- [30] scikitlearn developers. *sklearn.preprocessing.StandardScaler — scikitlearn documentation*. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html> (visitado 28-05-2025).
- [31] Sloan Digital Sky Survey Collaboration. *SQL in SkyServer*. SDSS SkyServer DR16 Documentation. 2024. URL: https://skyserver.sdss.org/dr16/en/help/docs/sql_help.aspx (visitado 29-05-2025).
- [32] Deepsandhya Shukla. *XOR Problem with Neural Networks: An Explanation for Beginners*. 2 de mayo de 2025. URL: <https://www.analyticsvidhya.com/blog/2024/01/xor-problem-with-neural-networks-an-explanation-for-beginners/#h-what-is-the-xor-problem> (visitado 30-05-2025).

Glosario

A continuación, se presentan los términos relevantes utilizados a lo largo de este TFG junto a la página donde aparecen por primera vez.

Marco Teórico

Aprendizaje automático (ML), 3
Modelo, 4
Dataset/Conjunto de datos, 4
Función de pérdida, 4
Pasos y épocas, 4
Validación, 4
Red Neuronal Artificial (RNA ó NN), 4
Parámetros y hiperparámetros, 5
Perceptrón, 5
Tasa de aprendizaje, 6
Perceptrón Multicapa (MLP), 6
Función de activación, 7
ReLU, 7
Retropropagación, 8
Optimizador, 8
Planificador, 9
Red neuronal convolucional (CNN), 10
Campo receptivo, 10
Kernel, 11
Stride, 12
Padding, 12
Pooling, 13
Transformers (TRA), 14
Embedding, 14

Atención, 14
Cabeza (de atención), 15
Overfitting, 16
Underfitting, 17
Dropout, 17
Decaimiento de pesos, 17

Redshift, 18
Ley de Hubble, 19
Espectros, 21

El Conjunto de Datos

Sloan Sky Digital Survey (SDSS), 23
Pipeline, 24
SQL, 26
FITS, 26
NED, 27
Cuásar, 29
Normalizador, 32

Modelización del Problema

Error absoluto medio (MAE), 36
Coeficiente de correlación lineal, 38

Resultados

Data drift, 51